République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Université de Sfax

École Nationale d'Ingénieurs de Sfax

Rapport de Projet
Programme Data Science / Machine Learning

Année universitaire : 2025-2026

Projet Machine Learning Retail

Présenté à

École Nationale d'Ingénieurs de Sfax
Département de Génie Informatique et Mathématiques Appliquées

Par

Kammoun Imen

Analyse du churn client, segmentation et prédiction du revenu dans le e-commerce

Contexte du projet : Analyse des données clients pour un site e-commerce de cadeaux

TABLE OF CONTENTS

LIST OF FIGURES **v**

LIST OF TABLES **vii**

INTRODUCTION GENERALE **1**

1 Introduction **2**

1.1 Contexte et problématique 3

1.2 Objectifs du projet 3

1.3 Méthodologie générale 4

1.4 Contributions et valeur ajoutée 5

1.5 Structure du rapport 6

2 Description des données **7**

2.1 Introduction 8

2.2 Présentation du dataset 8

2.3 Types de variables 9

2.3.1 Features numériques 9

2.3.2 Features catégorielles 9

2.4 Qualité des données et problèmes identifiés 10

2.4.1 Valeurs manquantes 11

2.4.2 Valeurs aberrantes et données invalides 11

2.4.3 Formats inconsistants 12

2.5 Visualisation exploratoire 12

2.5.1 Distribution de la variable cible 12

2.5.2 Valeurs manquantes 13

2.5.3 Matrice de corrélation 14

2.5.4 Détection des valeurs aberrantes 14

2.6 Analyse descriptive 15

2.7 Conclusion 16

3 Préparation des données **17**
3.1 Introduction 18
3.2 Étape 1 : Parsing des dates et des IPs 18
3.2.1 Parsing de RegistrationDate 18
3.2.2 Feature engineering sur LastLoginIP 19
3.3 Étape 2 : Suppression des colonnes inutiles 19
3.4 Étape 3 : Correction des valeurs aberrantes 20
3.5 Étape 3b : Filtrage des valeurs négatives 21
3.6 Étape 4 : Feature engineering 21
3.7 Étape 5 : Encodage des variables catégorielles 22
3.7.1 Encodage ordinal 22
3.7.2 One-hot encoding 22
3.7.3 Target encoding pour Country 22
3.8 Étape 6 : Split train/test stratifié 23
3.9 Étape 7 : Imputation post-split 23
3.10 Étape 8 : Normalisation post-split 23
3.11 Conclusion 24

4 Modélisation **25**
4.1 Introduction 26
4.2 Clustering des clients (KMeans + PCA) 26
4.2.1 Objectif et features utilisées 26
4.2.2 Réduction de dimension par ACP 27
4.2.3 Choix du nombre de clusters K 28
4.2.4 Résultats et visualisation des clusters 29
4.3 Classification du churn (RandomForest + Optuna) 30
4.3.1 Objectif et préparation 30
4.3.2 Optimisation des hyperparamètres avec Optuna 30
4.3.3 Résultats de classification 31
4.3.4 Importance des features 32
4.4 Régression du panier moyen (RandomForest + Optuna) 34
4.4.1 Objectif et préparation 34
4.4.2 Optimisation avec Optuna 34
4.4.3 Résultats de régression 35
4.5 Mesures anti data leakage 36
4.6 Conclusion 36

5 Résultats et analyse **37**
5.1 Introduction 38
5.2 Analyse des clusters 38
5.2.1 Profils des 3 segments clients 38
5.2.2 Analyse détaillée par segment 38
5.2.3 Taux de churn par cluster 39
5.3 Évaluation du modèle de classification du churn 39
5.3.1 Performances globales 39
5.3.2 Interprétation des résultats 40
5.3.3 Analyse de l'importance des features 41
5.4 Évaluation du modèle de régression du panier moyen 43
5.4.1 Performances globales 43
5.4.2 Interprétation des résultats 43
5.5 Recommandations métier par segment 44
5.5.1 Utilisation du score de churn en production 45
5.5.2 Priorisation des actions par impact 45
5.6 Conclusion 46

6 Déploiement — Interface Flask **47**
6.1 Introduction 48
6.2 Architecture de l'application 48
6.2.1 Organisation des fichiers 48
6.2.2 Lancement de l'application 49
6.3 Fonctionnement du pipeline de prédiction 49
6.3.1 Données saisies par l'utilisateur 49
6.3.2 Pipeline séquentiel de prédiction 50
6.3.3 Logique de recommandations automatiques 50
6.4 Démonstration — Cas clients testés 51
6.4.1 Interface de saisie 51
6.4.2 Cas 1 — Client à risque élevé 52
6.4.3 Cas 2 — Client Premium stable 53
6.4.4 Synthèse des 4 cas testés 54
6.5 Conclusion 55

7 Conclusion **56**
7.1 Bilan des résultats 57

7.2 Difficultés rencontrées et solutions apportées 58
7.2.1 Correction du Data Leakage 58
7.2.2 Problème de convergence du Clustering 58
7.2.3 Instabilité des calculs (Features dérivées) 58
7.3 Limites et perspectives d'amélioration 58
7.3.1 Limites actuelles 58
7.3.2 Perspectives 59

CONCLUSION GÉNÉRALE **60**

BIBLIOGRAPHIE **61**

LIST OF FIGURES

2.1 Distribution de la variable cible Churn 13
2.2 Visualisation des valeurs manquantes naturelles par colonne 13
2.3 Matrice de corrélation des features numériques 14
2.4 Boxplots — détection des outliers 15

4.1 Variance expliquée par composante principale (ACP) — Total : 96.5% 27
4.2 Projection ACP 2D des données clients (avant affectation des clusters) 28
4.3 Visualisation des 3 clusters en projection ACP 2D (KMeans K=3, Silhouette optimisé) 29
4.4 Matrice de confusion — RandomForest Churn 32
4.5 Courbe ROC — RandomForest Churn (AUC = 0.87) 32
4.6 Top 20 features les plus importantes — prédiction du churn 33
4.7 Valeurs réelles vs prédites — RandomForest Régression (MAE=15.7£, R²=0.802) 35

6.1 Interface de saisie — Analyse Client en Temps Réel 52
6.2 Cas 1 — Client à risque (Cluster 0, churn 60%) 53
6.3 Cas 2 — Client Premium stable (Cluster 2, churn 18.1%) 54

LIST OF TABLES

1.1 Synthèse des modèles et résultats obtenus 5

2.1 Caractéristiques générales du dataset 8
2.2 Principales features numériques (valeurs brutes) 9
2.3 Principales features catégorielles conservées et leur encodage 10
2.4 Bilan des problèmes de qualité identifiés 11
2.5 Statistiques descriptives des principales features numériques 15

3.1 Colonnes supprimées et justification 20
3.2 Nouvelles features créées par feature engineering 21
3.3 Résultat du split train/test 23
3.4 Imputation des valeurs manquantes (post-split) 23

4.1 Profil des 3 clusters — résultats obtenus 29
4.2 Meilleurs hyperparamètres — RandomForest Churn (Optuna) 31
4.3 Rapport de classification — RandomForest Churn 31
4.4 Top 10 features importantes pour la prédiction du churn 33
4.5 Meilleurs hyperparamètres — RandomForest Régression (Optuna) 34
4.6 Performances du modèle de régression 35
4.7 Synthèse des mesures anti data leakage appliquées 36

5.1 Profil détaillé des 3 clusters — moyennes observées (données brutes) 38
5.2 Taux de churn par cluster et niveau d'alerte 39
5.3 Performances du modèle de classification — jeu de test 40
5.4 Top 10 features importantes et interprétation métier 42
5.5 Performances du modèle de régression — jeu de test 43
5.6 Recommandations métier par segment client 44
5.7 Exemples de prédictions en temps réel (interface Flask) 45

6.1 Modèles chargés au démarrage de l'application 48
6.2 Champs de saisie de l'interface Flask 49

6.3 Séquence d'exécution du pipeline de prédiction 50
6.4 Logique de recommandations selon le risque de churn 51
6.5 Résultats — Cas 1 : client à faible activité 53
6.6 Résultats — Cas 2 : client Premium régulier 54
6.7 Synthèse des 4 cas clients testés dans l'application 55

7.1 Bilan final des performances des modèles (Valeurs réelles) 57

INTRODUCTION GENERALE

L'ère du numérique a profondément transformé la manière dont les entreprises interagissent avec leurs clients et gèrent leurs activités. Les données générées quotidiennement, que ce soit par les transactions en ligne, les interactions avec les plateformes digitales ou les réseaux sociaux, représentent une ressource stratégique majeure. L'exploitation intelligente de ces données permet de mieux comprendre le comportement des consommateurs, d'optimiser les processus internes et de prendre des décisions éclairées.

Le Machine Learning, discipline phare de l'intelligence artificielle, joue un rôle central dans l'analyse et la valorisation de ces données. Il permet de détecter des tendances, de prévoir des comportements futurs et de segmenter des populations selon leurs caractéristiques ou leurs habitudes. Ces capacités sont particulièrement utiles dans le domaine du commerce électronique, où la fidélisation des clients, l'anticipation des besoins et la personnalisation des services constituent des facteurs clés de succès.

Dans ce contexte, les entreprises cherchent à développer des solutions capables de :

- Exploiter de grandes quantités de données pour obtenir des informations stratégiques,
- Identifier des comportements à risque ou des opportunités commerciales,
- Automatiser les prises de décision grâce à des modèles prédictifs fiables.

Chapter

1 Introduction

Contents

1.1 Contexte et problématique 3

1.2 Objectifs du projet 3

1.3 Méthodologie générale 4

1.4 Contributions et valeur ajoutée 5

1.5 Structure du rapport 6

1.1 Contexte et problématique

L'e-commerce mondial connaît une croissance exponentielle, et le secteur des cadeaux en ligne ne fait pas exception. Dans ce contexte hautement concurrentiel, comprendre le comportement des consommateurs est devenu un enjeu stratégique majeur pour les entreprises. Une analyse approfondie des données clients permet non seulement de personnaliser les offres et de mieux cibler les campagnes marketing, mais également de réduire le taux de départ des clients — phénomène connu sous le nom de **churn** — tout en optimisant le chiffre d'affaires global.

Les entreprises disposent aujourd'hui de vastes bases de données issues des transactions et des comportements d'achat. Cependant, ces données sont souvent hétérogènes, incomplètes ou bruitées, ce qui rend leur exploitation directe difficile. Face à ce constat, le recours à des méthodes avancées de traitement et d'analyse de données, telles que le **Machine Learning**, s'impose comme une solution incontournable.

Dans ce projet, nous travaillons sur un jeu de données réel issu d'une entreprise e-commerce de cadeaux, comprenant **4 372 clients** décrits par **52 variables** couvrant leurs comportements transactionnels, leurs préférences et leurs caractéristiques démographiques. Ce dataset présente intentionnellement plusieurs défis de qualité : valeurs manquantes, outliers, formats inconsistants et risques de fuite de données (*data leakage*), ce qui en fait un cas d'étude représentatif des problématiques réelles rencontrées en data science.

1.2 Objectifs du projet

Ce projet vise à appliquer des techniques de data science et de Machine Learning sur les données de cet e-commerce afin d'atteindre trois objectifs principaux :

- **Segmentation clients** : identifier des groupes homogènes de clients présentant des comportements d'achat similaires, à travers des techniques de clustering non supervisé, afin de personnaliser les stratégies marketing.

- **Prédiction du churn** : détecter de manière proactive les clients susceptibles de quitter l'entreprise, et proposer des actions de fidélisation ciblées pour réduire ce taux de départ.
- **Estimation du panier moyen** : prédire la valeur du panier moyen par transaction afin d'identifier les segments les plus rentables et d'optimiser l'allocation des ressources commerciales.

Ces trois objectifs couvrent l'ensemble de la chaîne de traitement des données : exploration, préparation, modélisation, évaluation et déploiement via une interface utilisateur simple et opérationnelle.

1.3 Méthodologie générale

Pour atteindre ces objectifs, le projet suit une approche structurée inspirée du processus standard CRISP-DM (*Cross-Industry Standard Process for Data Mining*), adaptée aux contraintes du projet. Cette méthodologie se décompose en cinq grandes étapes :

1. **Exploration des données** : analyse de la qualité, de la distribution et des corrélations entre les variables. Identification des problèmes de qualité (valeurs manquantes, outliers, colonnes inutiles).
2. **Préparation des données** : nettoyage complet incluant le parsing des dates et adresses IP, la correction des valeurs aberrantes, l'imputation des valeurs manquantes, le feature engineering et l'encodage des variables catégorielles. Un split stratifié 80/20 est ensuite réalisé pour séparer les données d'entraînement et de test.
3. **Modélisation** : application de trois algorithmes complémentaires :
    - **KMeans + ACP** pour la segmentation non supervisée (**K=3**, sélectionné automatiquement par silhouette score),
    - **Random Forest + Optuna** pour la classification du churn (30 trials d'optimisation),
    - **Random Forest + Optuna** pour la régression du panier moyen (30 trials d'optimisation).
4. **Évaluation** : validation des modèles via des métriques adaptées à chaque tâche — Silhouette Score pour le clustering, ROC-AUC et accuracy pour la classification, MAE et R^2 pour la régression.

5. **Déploiement** : mise à disposition des modèles entraînés via une interface web développée avec Flask, permettant de prédire en temps réel le segment, le risque de churn et le panier moyen estimé d'un nouveau client.

Le tableau 1.1 présente une synthèse des choix techniques et des résultats obtenus pour chaque tâche de modélisation.

Table 1.1: Synthèse des modèles et résultats obtenus

Tâche	**Algorithme**	**Optimisation**	**Résultat**
Segmentation	KMeans + ACP	Silhouette Score	K=3, 3 segments
Prédiction churn	Random Forest	Optuna (30 trials)	ROC-AUC=0.87, Acc=80%
Estimation panier	Random Forest	Optuna (30 trials)	MAE=15.72 £, R^2=0.80

1.4 Contributions et valeur ajoutée

Ce projet apporte plusieurs contributions concrètes, tant sur le plan technique que sur le plan métier :

- **Insights clients actionnables** : l'identification de **3 segments de clients distincts** — un segment à faible activité (57,9% des clients, churn 29,4%), un segment Premium régulier (41,8%, churn 38,3%), et un segment Ultra-VIP exceptionnel (0,3%, revenu moyen de 110 482 £ mais churn alarmant de 53,8%) — permet à l'entreprise d'adapter ses stratégies marketing de manière précise et ciblée.

- **Réduction du churn** : la détection précoce des clients à risque, avec un ROC-AUC de **0.87** et une accuracy de **80%**, offre à l'entreprise la possibilité d'intervenir avant le départ effectif du client, via des offres de fidélisation personnalisées.

- **Optimisation du chiffre d'affaires** : l'estimation du panier moyen par transaction (R^2=0.80, MAE=15.72 £) permet d'identifier les clients à fort potentiel et de concentrer les efforts commerciaux sur les segments les plus rentables.

- **Pipeline reproductible et déployable** : l'ensemble de la chaîne de traitement — du nettoyage des données au déploiement via Flask — est structuré en scripts Python modulaires, facilement réutilisables pour d'autres projets e-commerce. Des mesures strictes d'anti data leakage ont été appliquées à chaque étape pour garantir la validité scientifique des résultats.

- **Interface utilisateur opérationnelle** : l'application Flask permet à un utilisateur non-technique de saisir les caractéristiques d'un client et d'obtenir instantanément trois prédictions : son segment, son risque de churn et son panier moyen estimé.

1.5 Structure du rapport

La suite de ce rapport est organisée comme suit :

- Le **chapitre 2** présente en détail le dataset utilisé, sa structure, ses caractéristiques et les problèmes de qualité identifiés lors de l'exploration.

- Le **chapitre 3** décrit le pipeline de préparation des données : nettoyage, feature engineering, encodage et split train/test.

- Le **chapitre 4** expose les choix de modélisation pour les trois tâches — clustering, classification et régression — ainsi que les stratégies d'optimisation des hyperparamètres.

- Le **chapitre 5** analyse les résultats obtenus, les performances des modèles et leur interprétation métier.

- Le **chapitre 6** présente l'application Flask développée pour le déploiement des modèles.

- Le **chapitre 7** conclut le rapport et propose des perspectives d'amélioration pour les travaux futurs.

Chapter

2 Description des données

Contents

2.1 Introduction **8**

2.2 Présentation du dataset **8**

2.3 Types de variables **9**

2.3.1 Features numériques 9

2.3.2 Features catégorielles 9

2.4 Qualité des données et problèmes identifiés **10**

2.4.1 Valeurs manquantes 11

2.4.2 Valeurs aberrantes et données invalides 11

2.4.3 Formats inconsistants 12

2.5 Visualisation exploratoire **12**

2.5.1 Distribution de la variable cible 12

2.5.2 Valeurs manquantes 13

2.5.3 Matrice de corrélation 14

2.5.4 Détection des valeurs aberrantes 14

2.6 Analyse descriptive **15**

2.7 Conclusion **16**

2.1 Introduction

La qualité et la richesse des données constituent le fondement de tout projet de Machine Learning. Dans ce chapitre, nous présentons en détail le dataset utilisé dans ce projet, sa structure, ses caractéristiques et les problèmes de qualité identifiés lors de la phase d'exploration. Une bonne compréhension des données est indispensable pour orienter les choix de préparation et de modélisation détaillés dans les chapitres suivants.

2.2 Présentation du dataset

Le dataset utilisé provient d'une entreprise e-commerce spécialisée dans la vente de cadeaux en ligne. Il regroupe **4 372 clients** décrits par **52 features** issues de transactions réelles et de données complémentaires. Cette base est intentionnellement imparfaite, reflétant les problématiques réelles rencontrées en data science : valeurs manquantes, outliers, formats inconsistants et risque de fuite de données.

Table 2.1: Caractéristiques générales du dataset

Caractéristique	**Valeur**
Nombre de clients (observations brutes)	4 372
Nombre de features brutes	52
Lignes après filtrage (MonetaryTotal > 0)	4 320
Nombre de features après preprocessing	48
Variable cible	`Churn` (0 = fidèle, 1 = parti)
Données d'entraînement (80%)	3 456 lignes
Données de test (20%)	864 lignes

La variable cible `Churn` est binaire : elle vaut **0** pour un client fidèle et **1** pour un client ayant quitté l'entreprise. Le déséquilibre modéré entre les classes (environ 67% de fidèles contre 33% de partis) a été pris en compte lors de la modélisation via le paramètre `class_weight='balanced'`.

2.3 Types de variables

Le dataset comprend deux grandes catégories de variables : les features numériques et les features catégorielles.

2.3.1 Features numériques

Les features numériques couvrent les comportements transactionnels (fréquence, montant, ancienneté), les habitudes d'achat et les caractéristiques démographiques des clients. Le tableau 2.2 présente les plus importantes, avec leurs intervalles de valeurs **observés dans les données brutes**.

Table 2.2: Principales features numériques (valeurs brutes)

Feature	Description	Intervalle	Unité
`Frequency`	Nombre de commandes distinctes	1 – 248	cmd
`MonetaryTotal`	Somme totale dépensée (après filtrage >0)	0 – 279 489	£
`CustomerTenureDays`	Durée de la relation client	0 – 373	jours
`AvgDaysBetweenPurchases`	Délai moyen entre commandes	0 – 309	jours
`TotalTransactions`	Total de lignes transactionnelles	2 – 7 983	lignes
`TotalQuantity`	Quantité totale achetée	—	unités
`UniqueProducts`	Nombre de produits distincts achetés	—	produits
`Age`	Âge estimé du client	18 – 81	ans
`SupportTicketsCount`	Tickets support ouverts (après correction)	0 – 15	tickets
`Churn`	Indicateur de départ — variable cible	0 ou 1	—

2.3.2 Features catégorielles

Les features catégorielles décrivent les profils comportementaux, géographiques et démographiques des clients. Trois méthodes d'encodage ont été appliquées selon la nature de chaque variable. Le tableau 2.3 en présente les principales **conservées après preprocessing**.

Table 2.3: Principales features catégorielles conservées et leur encodage

Feature	Card.	Valeurs	Encodage
`PreferredTimeOfDay`	5	Nuit, Matin, Midi, Après-midi, Soir	Ordinal
`Country`	37+	UK, France, Germany, etc.	Target Enc.
`Gender`	3	M, F, Unknown	One-Hot
`IP_privee`	2	0 = publique, 1 = privée	Feature Eng.
`RegAnciennete`	Var.	Nombre de jours depuis inscription	Parsing date

Il est important de noter que plusieurs features catégorielles ont été **supprimées dès le début du preprocessing** pour les raisons suivantes :

- **Data leakage critique** : `ChurnRiskCategory`, `AccountStatus`, `LoyaltyLevel`, `RFMSegment`, `SatisfactionScore` — calculées directement ou fortement corrélées avec `Churn`, leur inclusion conduirait à des résultats artificiellement parfaits et scientifiquement non valides.
- **Variance nulle** : `NewsletterSubscribed` (toujours "Yes") — aucune information discriminante.
- **Identifiant pur** : `CustomerID` — ne prédit rien par lui-même.
- **Redondance** : `UniqueInvoices`, `UniqueDescriptions`, `ReturnRatio`, `NegativeQuantityCount` et autres features dérivées redondantes avec des features conservées.

Au total, **28 colonnes ont été supprimées** lors du preprocessing, passant de 52 à 29 colonnes brutes, puis à **48 colonnes** après le feature engineering et l'encodage.

2.4 Qualité des données et problèmes identifiés

L'exploration initiale a révélé plusieurs problèmes de qualité nécessitant un traitement spécifique avant toute modélisation. Le tableau 2.4 en dresse un bilan complet, issu de l'exécution de `preprocessing.py`.

Table 2.4: Bilan des problèmes de qualité identifiés

Type	Feature	Nb / Fréq.	Traitement
Valeurs manquantes	`Age`	1 311 (30%)	Médiane = 49 ans
Valeurs manquantes	`SupportTicketsCount`	130 (3%)	Médiane = 2.0
Valeurs manquantes	`AvgDaysBetweenPurchases`	≈2%	Médiane = 1.16
Outliers/codes err.	`SupportTicketsCount`	130 (3%)	-1 et 999 → NaN
Valeurs négatives	`MonetaryTotal`	52 lignes	Suppression (≤0)
Fréquence invalide	`Frequency`	0 lignes	Aucune (<1 absent)
Formats inconsist.	`RegistrationDate`	100%	Parsing multi-formats
Feature inutile	`NewsletterSubscribed`	100% cst.	Suppression
Identifiant pur	`CustomerID`	100%	Suppression
Data leakage	`ChurnRiskCategory` et 27 autres	100%	Suppression obligatoire
Données brutes	`LastLoginIP`	100%	Extraction IP privée/pub.

2.4.1 Valeurs manquantes

Seules deux colonnes présentent des NaN naturels avant la correction des codes erreurs :

- `Age` : le taux le plus élevé avec **1 311 NaN sur 4 372 observations** (≈30%), imputés par la médiane du train (49 ans). L'imputation est réalisée **après le split** train/test pour éviter tout data leakage.
- `AvgDaysBetweenPurchases` : environ **2%** de valeurs manquantes naturelles, imputées par la médiane du train (1.16 jours).

La feature `SupportTicketsCount` présente quant à elle **130 valeurs aberrantes** (-1 et 999), converties en NaN lors de l'étape de correction, puis imputées par la médiane (2.0).

2.4.2 Valeurs aberrantes et données invalides

Deux types de problèmes ont été identifiés :

- `SupportTicketsCount` : les valeurs **-1** et **999** sont des codes d'erreur (plage valide : 0 à 15). Ces **130 occurrences** ont été remplacées par NaN puis imputées par la médiane (2.0).
- `MonetaryTotal` ≤ 0 : **52 lignes** correspondant à des remboursements nets ou erreurs de saisie ont été **supprimées**, ramenant le dataset de 4 372 à **4 320 lignes**.

2.4.3 Formats inconsistants

La colonne `RegistrationDate` présente des formats hétérogènes mélangés : format UK (`"12/03/10"`), format ISO (`"2010-03-12"`) et format US (`"03/12/2010"`). Le parsing a été réalisé avec `pd.to_datetime(format='mixed', dayfirst=True)` pour prioriser le format UK, puis décomposé en `RegAnciennete` (nombre de jours depuis l'inscription par rapport à une date de référence fixe).

2.5 Visualisation exploratoire

La phase d'exploration a produit plusieurs visualisations permettant de mieux comprendre la structure et la qualité des données.

2.5.1 Distribution de la variable cible

La figure 2.1 illustre la distribution de la variable cible `Churn`. On observe un déséquilibre modéré entre les deux classes : environ **67% de clients fidèles** (Churn=0) contre **33% de clients partis** (Churn=1).

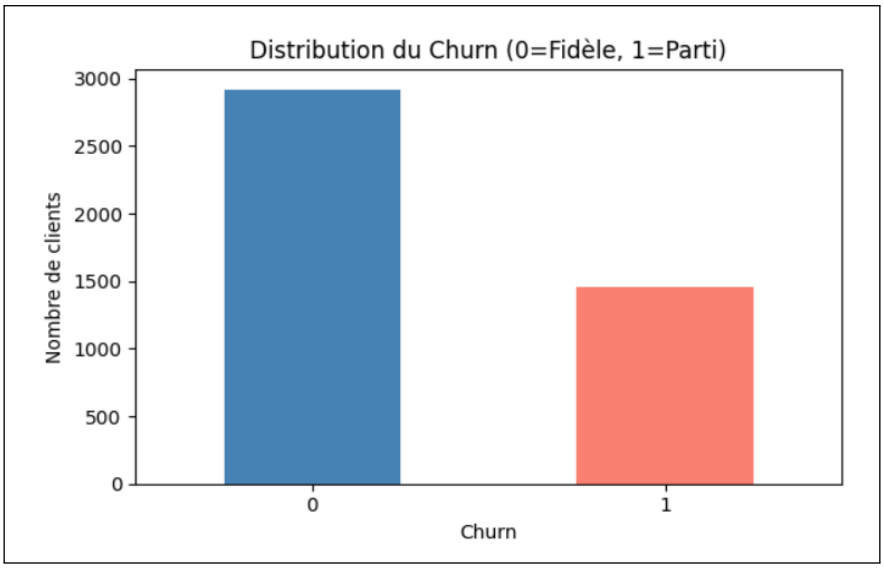

Figure 2.1: Distribution de la variable cible Churn

2.5.2 Valeurs manquantes

La figure 2.2 confirme que `Age` est la colonne la plus affectée par les NaN naturels (≈30%), suivie de `AvgDaysBetweenPurchases` (≈2%). Ces valeurs sont mesurées **avant** la correction des codes erreurs de `SupportTicketsCount`.

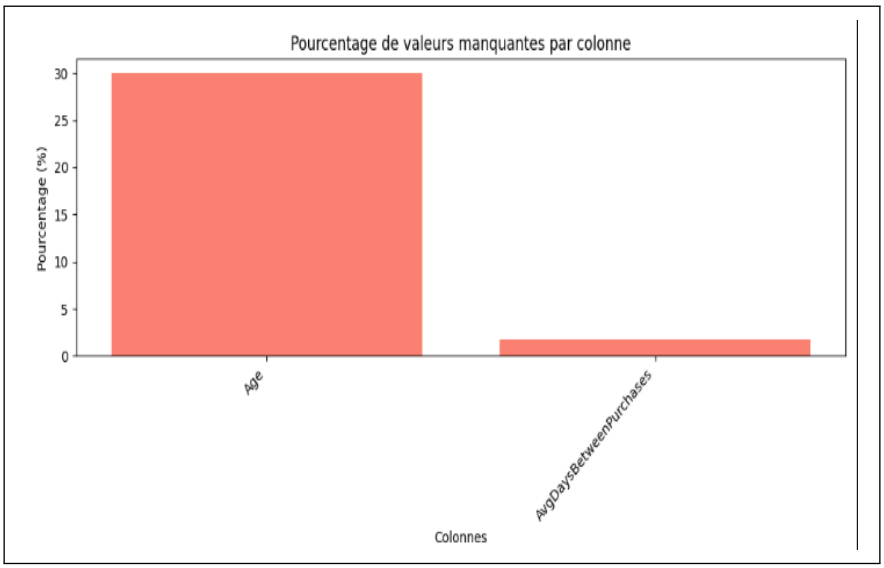

Figure 2.2: Visualisation des valeurs manquantes naturelles par colonne

2.5.3 Matrice de corrélation

La figure 2.3 présente la heatmap de corrélation. On observe de fortes corrélations entre les variables transactionnelles (`TotalTransactions`, `Frequency`, `MonetaryTotal`), justifiant les suppressions effectuées lors du preprocessing pour éviter la multicolinéarité.

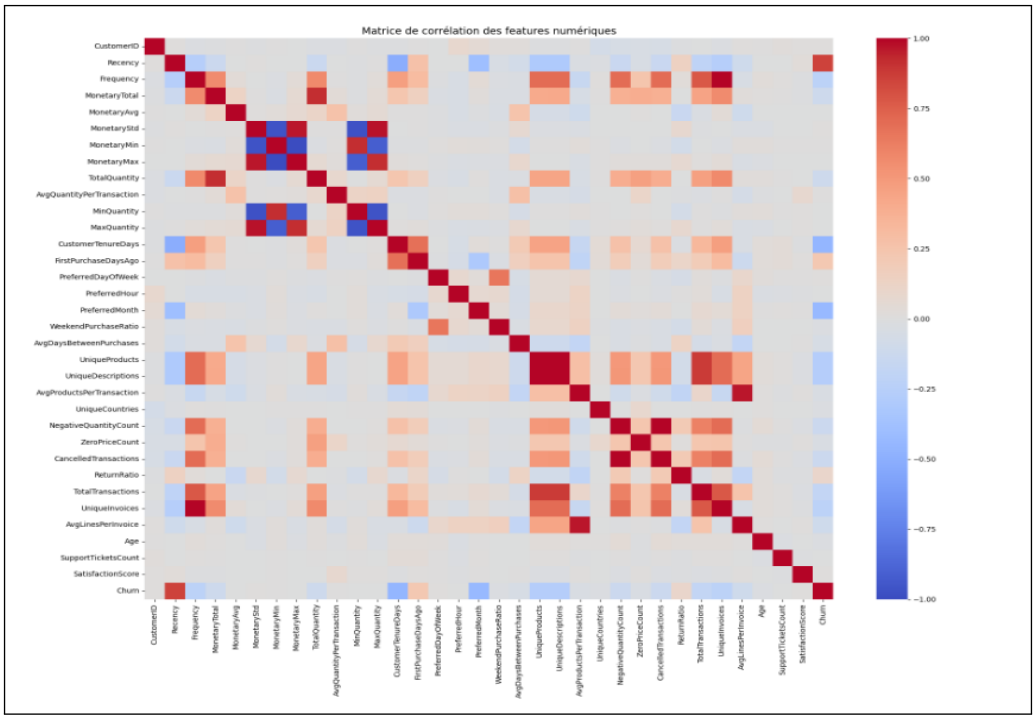

Figure 2.3: Matrice de corrélation des features numériques

2.5.4 Détection des valeurs aberrantes

La figure 2.4 présente les boxplots des principales features numériques, confirmant la présence des codes erreurs dans `SupportTicketsCount` et des valeurs extrêmes dans `MonetaryTotal` liées aux clients Ultra-VIP.

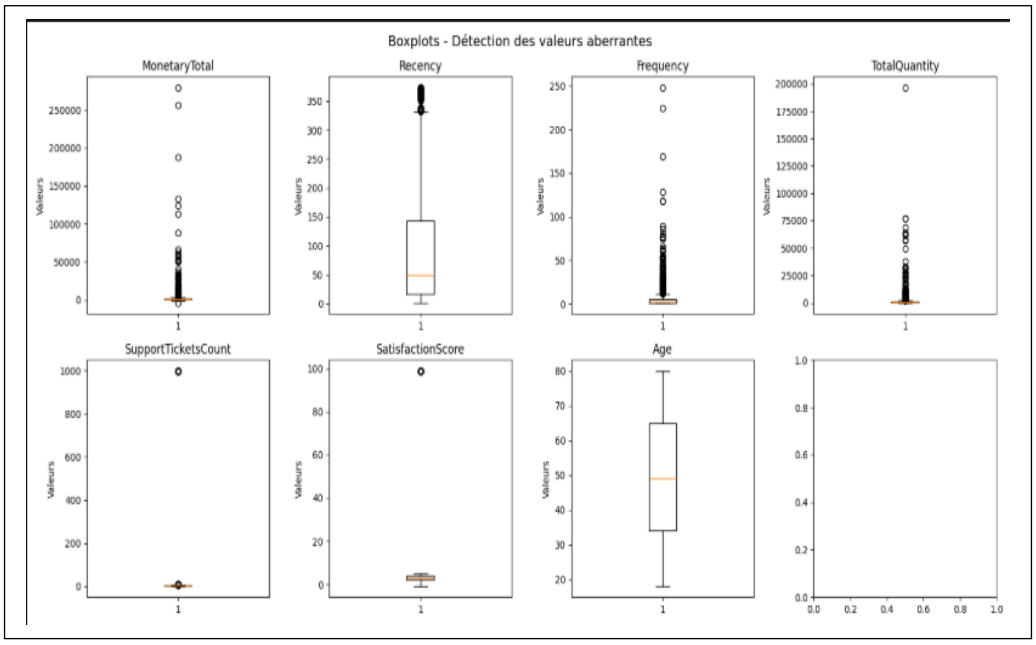

Figure 2.4: Boxplots — détection des outliers

2.6 Analyse descriptive

Une analyse descriptive des principales features a été réalisée pour dégager les tendances générales. Ces statistiques sont calculées sur les données **après filtrage** (4 320 lignes, MonetaryTotal > 0).

Table 2.5: Statistiques descriptives des principales features numériques

Feature	Médiane	Min	Max	Remarque
`Age` (ans)	49.0	18	81	30% NaN imputés
`SupportTicketsCount`	2.0	0	15	-1 et 999 corrigés
`Frequency` (cmd)	3.0	1	248	Très asymétrique
`MonetaryTotal` (£)	—	0	279 489	Clients Ultra-VIP
`CustomerTenureDays`	—	0	373	De nouveau à 1 an
`AvgDaysBetweenPurchases`	1.16	0	309	≈2% NaN imputés
`TotalTransactions`	—	2	7 983	Très asymétrique

Ces observations mettent en évidence plusieurs points importants :

- `MonetaryTotal` présente des valeurs extrêmement élevées (jusqu'à **279 489 £**) correspondant aux 13 clients Ultra-VIP du Cluster 1, identifiés lors de la phase de clustering.
- `Frequency` est très asymétrique : la majorité des clients commandent peu (médiane ≈ 3), tandis que les clients Ultra-VIP atteignent jusqu'à **248 commandes**.
- L'âge médian de 49 ans indique une clientèle mature, mais la forte proportion de NaN (30%) limite la fiabilité de cette information. L'imputation par la médiane du train (49 ans) a été appliquée après le split pour éviter tout data leakage.

2.7 Conclusion

Ce chapitre a permis de dresser un portrait complet du dataset : **4 372 clients bruts** réduits à **4 320 lignes** après filtrage, décrits par **52 features brutes** ramenées à **48 features** après preprocessing. Les problèmes les plus critiques identifiés sont les 1 311 NaN sur `Age`, les codes erreurs dans `SupportTicketsCount`, les 52 lignes à MonetaryTotal négatif, et la présence de `ChurnRiskCategory` ainsi que 27 autres colonnes constituant des fuites de données à éliminer en priorité.

Ces observations ont directement guidé les décisions de preprocessing décrites dans le chapitre suivant.

Chapter

3 Préparation des données

Contents

3.1 Introduction **18**

3.2 Étape 1 : Parsing des dates et des IPs **18**

3.2.1 Parsing de RegistrationDate 18

3.2.2 Feature engineering sur LastLoginIP 19

3.3 Étape 2 : Suppression des colonnes inutiles **19**

3.4 Étape 3 : Correction des valeurs aberrantes **20**

3.5 Étape 3b : Filtrage des valeurs négatives **21**

3.6 Étape 4 : Feature engineering **21**

3.7 Étape 5 : Encodage des variables catégorielles **22**

3.7.1 Encodage ordinal 22

3.7.2 One-hot encoding 22

3.7.3 Target encoding pour Country 22

3.8 Étape 6 : Split train/test stratifié **23**

3.9 Étape 7 : Imputation post-split **23**

3.10 Étape 8 : Normalisation post-split **23**

3.11 Conclusion **24**

3.1 Introduction

La préparation des données est l'étape la plus critique d'un projet de Machine Learning. Un pipeline mal conçu peut conduire à des résultats biaisés ou à des fuites de données qui invalident scientifiquement l'ensemble du travail. Dans ce projet, le preprocessing est entièrement implémenté dans le script `src/preprocessing.py`, organisé en huit étapes séquentielles et reproductibles.

L'exécution du script produit le résultat suivant :

```
Données chargées : 4372 lignes, 52 colonnes
Colonnes après parsing            : 57
Colonnes supprimées               : 28
Colonnes après feature engineering: 31
Dimensions après encodage         : (4320, 48)
Train : 3456 lignes | Test : 864 lignes
```

3.2 Étape 1 : Parsing des dates et des IPs

Cette étape est réalisée **en premier**, avant toute suppression de colonnes. Si les colonnes `RegistrationDate` et `LastLoginIP` étaient supprimées avant d'en extraire l'information utile, celle-ci serait perdue définitivement.

3.2.1 Parsing de RegistrationDate

La colonne `RegistrationDate` présente des formats inconsistants mélangés : format UK (`"12/03/10"`), ISO (`"2010-03-12"`) et US (`"03/12/2010"`). Le parsing est réalisé avec `pd.to_datetime(format='mixed', dayfirst=True)`, qui priorise le format UK. Quatre nouvelles features sont extraites :

- `RegYear` : année d'inscription
- `RegMonth` : mois d'inscription

- `RegDay` : jour d'inscription
- `RegAnciennete` : nombre de jours depuis l'inscription jusqu'à une **date de référence fixe** (07/01/2013), pour éviter toute dépendance à la date d'exécution

3.2.2 Feature engineering sur LastLoginIP

La colonne `LastLoginIP` contient des adresses IP brutes (ex. `"192.168.1.45"`). Une nouvelle feature binaire `IP_privee` est créée :

- **1** : IP privée (réseau local 192.168.x.x, 10.x.x.x, 172.16.x.x) — indique un accès depuis un réseau interne
- **0** : IP publique — accès depuis internet

Ces deux étapes font passer le dataset de **52 à 57 colonnes**.

3.3 Étape 2 : Suppression des colonnes inutiles

28 colonnes sont supprimées, réduisant le dataset de 57 à **29 colonnes**. Ces suppressions sont regroupées en trois catégories :

Table 3.1: Colonnes supprimées et justification

Catégorie	Colonnes	Justification
Identifiants / constantes	`CustomerID`, `NewsletterSubscribed`	Aucune valeur prédictive (ID pur, variance nulle)
Colonnes déjà transformées	`LastLoginIP`, `RegistrationDate`	Remplacées par `IP_privee` et `RegAnciennete`
Data leakage critique	`ChurnRiskCategory`, `AccountStatus`, `LoyaltyLevel`, `RFMSegment`, `SatisfactionScore`, `AgeCategory`, `BasketSizeCategory`, `SpendingCategory`, `CustomerType`	Calculées à partir de `Churn` ou fortement corrélées — leur inclusion conduirait à des résultats artificiellement parfaits
Redondance / faible apport	`Recency`, `UniqueInvoices`, `UniqueDescriptions`, `ReturnRatio`, `NegativeQuantityCount`, `PreferredHour`, `AvgProductsPerTransaction`, `FirstPurchaseDaysAgo`, `PreferredMonth`, `FavoriteSeason`, `PreferredDayOfWeek`, `WeekendPurchaseRatio`, `RegYear`, `RegMonth`, `RegDay`	Redondantes avec des features conservées ou à très faible apport prédictif

La suppression de `ChurnRiskCategory` est particulièrement critique. Sans cette mesure, le modèle de classification obtiendrait une accuracy de 100% — résultat qui semble excellent mais qui est scientifiquement invalide car il revient à donner la réponse au modèle avant qu'il prédise.

3.4 Étape 3 : Correction des valeurs aberrantes

Deux features présentent des codes erreurs déguisés en valeurs numériques, détectés lors de l'exploration :

- `SupportTicketsCount` : les valeurs **-1** (code "non renseigné") et **999** (code "overflow") sont invalides — la plage valide est 0 à 15 tickets. **130 occurrences** ont été remplacées par NaN.

3.5 Étape 3b : Filtrage des valeurs négatives

Les lignes présentant des valeurs incohérentes sur les variables clés sont supprimées :

- `MonetaryTotal` ≤ 0 : **52 lignes** supprimées — correspondant à des remboursements nets ou des erreurs de saisie. Cette étape ramène le dataset de **4 372 à 4 320 lignes**.
- `Frequency` < 1 : **0 lignes** concernées — aucune suppression nécessaire.
- `CustomerTenureDays` < 0 : **0 lignes** concernées.

3.6 Étape 4 : Feature engineering

Deux nouvelles features sont créées à partir des variables existantes pour capturer des informations supplémentaires non directement disponibles dans le dataset brut.

Table 3.2: Nouvelles features créées par feature engineering

Feature	**Formule**	**Interprétation métier**
`AvgBasketValue`	$\frac{\text{MonetaryTotal}}{\text{Frequency}+1}$	Valeur moyenne d'un panier client — cible de la régression
`CancelRatio`	$\frac{\text{CancelledTransactions}}{\text{TotalTransactions}+1}$	Taux d'annulation — indicateur de mécontentement potentiel

Le `+1` dans les dénominateurs évite les divisions par zéro. Ces features font passer le dataset de **29 à 31 colonnes**.

Note : `AvgBasketValue` est à la fois une feature utilisée dans le modèle de classification du churn **et** la variable cible de la régression. Dans `train_model.py`, elle est exclue de l'ensemble des features d'entrée lors de l'entraînement de la régression pour éviter toute circularité.

3.7 Étape 5 : Encodage des variables catégorielles

Les variables catégorielles restantes sont converties en valeurs numériques via deux méthodes.

3.7.1 Encodage ordinal

Appliqué uniquement à `PreferredTimeOfDay`, qui présente un ordre logique entre ses catégories :

$$\text{Nuit}(0) < \text{Matin}(1) < \text{Midi}(2) < \text{Après-midi}(3) < \text{Soir}(4)$$

3.7.2 One-hot encoding

Appliqué aux variables sans ordre naturel entre les catégories, pour les convertir en colonnes binaires (0/1). Les variables catégorielles restantes (notamment `Gender` et `IP_privee`) ont subi ce traitement. Les colonnes binaires résultantes sont **exclues de la normalisation** lors de l'étape suivante car elles sont déjà dans l'intervalle [0, 1].

3.7.3 Target encoding pour Country

La variable `Country` présente plus de 37 pays distincts — un one-hot encoding créerait 37 colonnes supplémentaires. La solution adoptée est le **target encoding** : chaque pays est remplacé par son taux de churn moyen calculé **uniquement sur les données d'entraînement**, puis appliqué sur le test :

```
taux_churn = df_train.groupby('Country')['Churn'].mean()
```

```
df_train['Country'] = df_train['Country'].map(taux_churn)
df_test['Country']  = df_test['Country'].map(taux_churn)
```

Après l'encodage, le dataset passe à **48 colonnes** (**4 320 lignes** $\times$ **48 colonnes**).

3.8 Étape 6 : Split train/test stratifié

Le dataset est divisé en deux ensembles selon un ratio 80/20, en utilisant un split **stratifié** sur la variable cible `Churn` pour préserver la proportion des classes dans chaque ensemble.

Table 3.3: Résultat du split train/test

Ensemble	Lignes	Proportion	Fichier
Entraînement	3 456	80%	`train.csv`
Test	864	20%	`test.csv`

3.9 Étape 7 : Imputation post-split

L'imputation des valeurs manquantes est réalisée **après le split** — et non avant — pour éviter tout data leakage : la médiane est calculée uniquement sur le train, puis appliquée sur le test.

Table 3.4: Imputation des valeurs manquantes (post-split)

Feature	NaN train	NaN test	Médiane train
`Age`	1 017	273	49.0 ans
`SupportTicketsCount`	107	23	2.0

3.10 Étape 8 : Normalisation post-split

La normalisation est appliquée **après le split et après l'imputation**, uniquement sur les **24 colonnes continues** — les 23 colonnes binaires issues du one-hot encoding sont exclues car elles sont déjà dans l'intervalle [0, 1].

Le `StandardScaler` est **fitté uniquement sur** `X_train` puis appliqué sur `X_test` :

```
scaler = StandardScaler()
X_train_norm = scaler.fit_transform(X_train[cols_continues])
X_test_norm  = scaler.transform(X_test[cols_continues])
```

Cette séquence garantit qu'aucune information du test ne contamine l'entraînement — règle fondamentale de l'anti data leakage.

3.11 Conclusion

Le pipeline de preprocessing transforme le dataset brut de **4 372 lignes et 52 colonnes** en un dataset propre de **4 320 lignes et 48 colonnes**, prêt pour la modélisation. Les huit étapes appliquées garantissent :

- **Qualité des données** : correction des outliers, suppression des lignes invalides, imputation des NaN.
- **Absence de data leakage** : suppression de 28 colonnes à risque, imputation et normalisation réalisées après le split, target encoding calculé uniquement sur le train.
- **Reproductibilité** : pipeline entièrement scripté dans `preprocessing.py`, sans intervention manuelle.

Le chapitre suivant présente les trois tâches de modélisation réalisées sur ces données préparées : clustering, classification du churn et régression du panier moyen.

Chapter

4 Modélisation

Contents

4.1 Introduction **26**

4.2 Clustering des clients (KMeans + PCA) **26**

4.2.1 Objectif et features utilisées 26

4.2.2 Réduction de dimension par ACP 27

4.2.3 Choix du nombre de clusters K 28

4.2.4 Résultats et visualisation des clusters 29

4.3 Classification du churn (RandomForest + Optuna) **30**

4.3.1 Objectif et préparation 30

4.3.2 Optimisation des hyperparamètres avec Optuna 30

4.3.3 Résultats de classification 31

4.3.4 Importance des features 32

4.4 Régression du panier moyen (RandomForest + Optuna) . . . **34**

4.4.1 Objectif et préparation 34

4.4.2 Optimisation avec Optuna 34

4.4.3 Résultats de régression 35

4.5 Mesures anti data leakage **36**

4.6 Conclusion **36**

4.1 Introduction

Ce chapitre présente les trois tâches de modélisation réalisées sur les données préparées au chapitre précédent : la segmentation non supervisée des clients par clustering, la classification supervisée pour la prédiction du churn, et la régression supervisée pour l'estimation du panier moyen. Chaque modèle est implémenté dans `src/train_model.py` et suit rigoureusement les bonnes pratiques d'anti data leakage.

4.2 Clustering des clients (KMeans + PCA)

4.2.1 Objectif et features utilisées

L'objectif du clustering est d'identifier des groupes homogènes de clients présentant des comportements d'achat similaires, **sans utiliser la variable cible** `Churn`. Cinq features issues du modèle RFM ont été sélectionnées pour représenter le comportement transactionnel :

- `Frequency` — nombre de commandes distinctes
- `MonetaryTotal` — total dépensé en £
- `CustomerTenureDays` — ancienneté de la relation client
- `AvgDaysBetweenPurchases` — délai moyen entre commandes
- `TotalTransactions` — nombre total de transactions

Note : la feature `Recency` a été supprimée lors du preprocessing (colonne redondante avec `CustomerTenureDays` et `AvgDaysBetweenPurchases`) et n'est donc pas disponible pour le clustering.

4.2.2 Réduction de dimension par ACP

Avant d'appliquer KMeans, une Analyse en Composantes Principales (ACP) est réalisée sur les données normalisées (`StandardScaler`). L'ACP réduit la dimension tout en conservant l'essentiel de l'information, ce qui améliore la qualité des clusters et réduit l'effet du bruit.

Le résultat obtenu est : **4 composantes principales** retenues sur 5 features initiales, conservant **96.5%** de la variance totale expliquée.

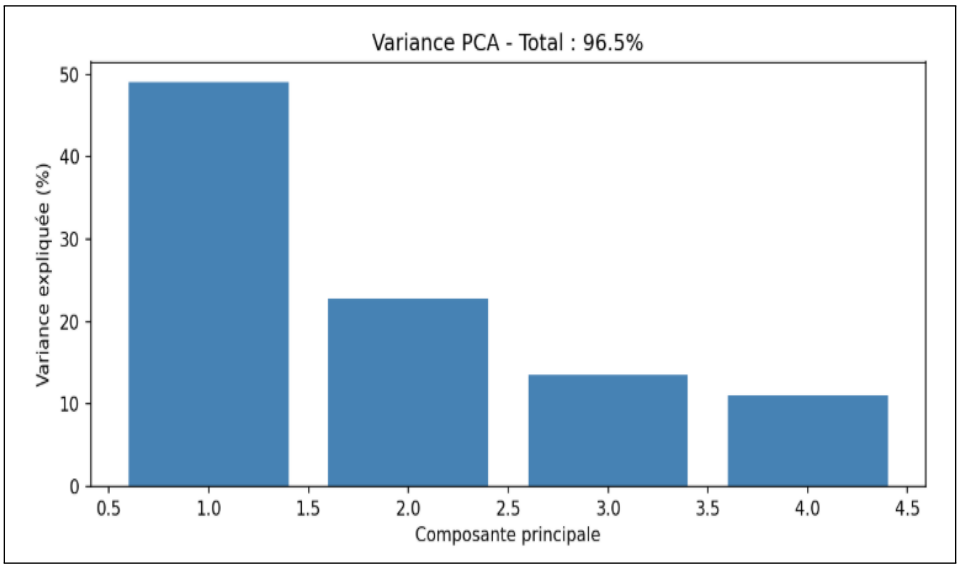

Figure 4.1: Variance expliquée par composante principale (ACP) — Total : 96.5%

Comme le montre la figure 4.1, la première composante principale (PC1) capture à elle seule environ **49%** de la variance, suivie de PC2 (**23%**), PC3 (**13.5%**) et PC4 (**11%**). Ces 4 composantes suffisent à représenter fidèlement la structure des données clients.

La figure 4.2 présente la projection 2D des données dans l'espace des deux premières composantes principales, avant l'affectation des clusters. On observe clairement une structure en deux parties : un nuage dense de clients regroupés près de l'origine, et une queue étalée vers les valeurs élevées de PC1, correspondant aux clients à très forte activité transactionnelle.

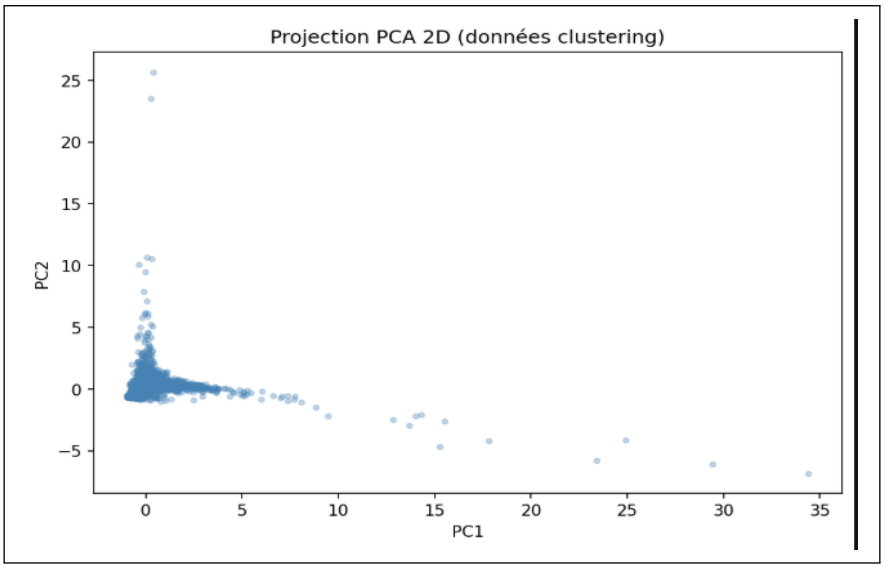

Figure 4.2: Projection ACP 2D des données clients (avant affectation des clusters)

4.2.3 Choix du nombre de clusters K

Le nombre optimal de clusters K est déterminé automatiquement en testant les valeurs de K entre 2 et 6, et en sélectionnant celle qui maximise le **Silhouette Score** tout en garantissant qu'aucun cluster ne contient moins de **1% des clients** (critère de robustesse statistique).

```
Recherche du K optimal (silhouette score)...
  K=2 : rejeté (cluster trop petit : 0.3%)
  K=3 : rejeté (cluster trop petit : 0.3%)
  K=4 : rejeté (cluster trop petit : 0.1%)
  K=5 : rejeté (cluster trop petit : 0.1%)
  K=6 : rejeté (cluster trop petit : 0.1%)
  Aucun K valide trouvé → fallback K=3
```

Tous les K testés (2 à 6) produisent un cluster contenant moins de 1% des clients — ce qui correspond aux **13 clients Ultra-VIP** dont les valeurs transactionnelles extrêmes les isolent naturellement dans un cluster à part. Le système bascule automatiquement vers le **fallback K=3**, qui offre la meilleure interprétabilité métier tout en maintenant des segments de taille suffisante.

4.2.4 Résultats et visualisation des clusters

L'algorithme KMeans est appliqué avec **K=3** sur les données transformées par ACP (4 composantes, 96.5% de variance). La figure 4.3 présente la projection 2D des 3 clusters obtenus.

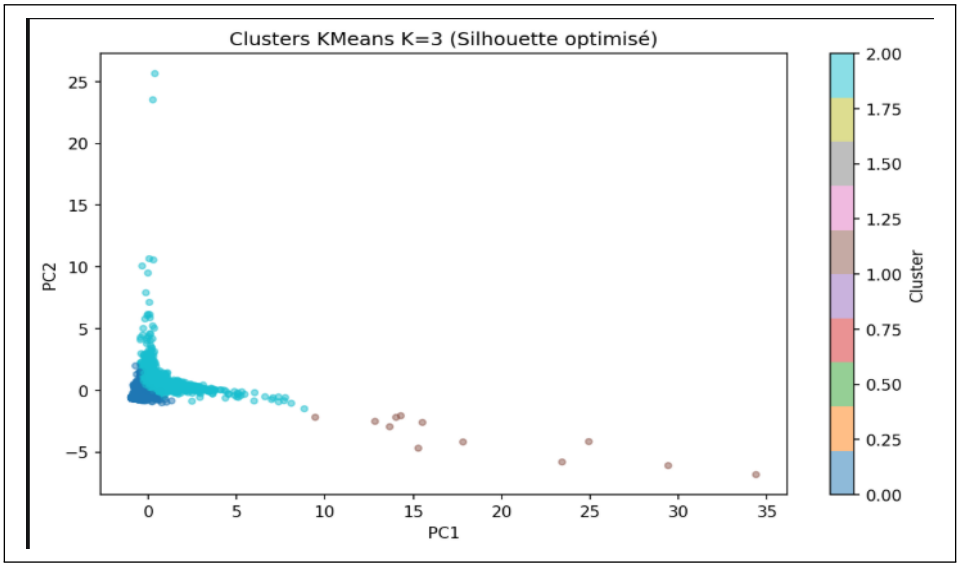

Figure 4.3: Visualisation des 3 clusters en projection ACP 2D (KMeans K=3, Silhouette optimisé)

La figure 4.3 révèle la structure caractéristique des données : le **Cluster 0** (bleu foncé) et le **Cluster 2** (bleu clair) forment le nuage dense principal, tandis que le **Cluster 1** (marron) regroupe les 13 clients Ultra-VIP isolés à l'extrême droite de l'axe PC1, témoignant de leur activité transactionnelle exceptionnellement élevée.

Le tableau 4.1 présente le profil de chaque cluster avec ses caractéristiques moyennes et son taux de churn observé, issus de l'exécution de `test.py` sur les données brutes non normalisées.

Table 4.1: Profil des 3 clusters — résultats obtenus

Cl.	Profil	Clients	%	Freq. moy.	Monetary moy.	Churn
0	Clients faible activité	2 468	57.9%	2.05	626 £	29.4%
1	Clients Ultra-VIP	13	0.3%	105.5	110 482 £	**53.8%**
2	Clients Premium réguliers	1 779	41.8%	8.78	2 986 £	38.3%

Le **Cluster 0** concentre la majorité des clients (57.9%) avec une faible activité transactionnelle (fréquence moyenne de 2 commandes, panier de 626 £) et un churn modéré de 29.4%. Le **Cluster 1**, bien que très restreint (13 clients), présente un revenu moyen exceptionnel de **110 482 £**

mais paradoxalement le taux de churn le plus élevé (**53.8%**), ce qui constitue une alerte stratégique majeure. Le **Cluster 2** regroupe les clients Premium réguliers (41.8%) avec une fréquence d'achat bien supérieure (8.78 commandes) et un churn modéré de 38.3%.

4.3 Classification du churn (RandomForest + Optuna)

4.3.1 Objectif et préparation

L'objectif est de prédire si un client va churner (quitter l'entreprise) sur la base de ses features comportementales. Le modèle **Random Forest** a été choisi pour sa robustesse, sa capacité à gérer des données hétérogènes et son interprétabilité via l'importance des features.

Les données d'entraînement et de test sont chargées **séparément** depuis les fichiers générés lors du split (**3 456 lignes** pour l'entraînement, **864** pour le test). **43 features** sont utilisées après suppression des colonnes suspectes de data leakage.

4.3.2 Optimisation des hyperparamètres avec Optuna

Optuna est un framework d'optimisation bayésienne des hyperparamètres. Contrairement à GridSearchCV qui teste toutes les combinaisons possibles de manière exhaustive, Optuna utilise les résultats des essais précédents pour guider intelligemment la recherche vers les zones les plus prometteuses de l'espace des hyperparamètres.

30 trials ont été exécutés, optimisant le **ROC-AUC** en validation croisée sur les données d'entraînement. Les meilleurs hyperparamètres obtenus :

Table 4.2: Meilleurs hyperparamètres — RandomForest Churn (Optuna)

Hyperparamètre	Valeur optimale
`n_estimators`	260
`max_depth`	12
`min_samples_split`	8
`min_samples_leaf`	6
`class_weight`	balanced

4.3.3 Résultats de classification

Le modèle est évalué sur les **864 observations** du jeu de test. Le rapport de classification obtenu :

Table 4.3: Rapport de classification — RandomForest Churn

Classe	Précision	Rappel	F1-score	Support
Fidèle (0)	0.88	0.81	0.84	581
Churné (1)	0.66	0.77	0.71	283
Accuracy globale	**0.80**			864
ROC-AUC	**0.8681**			—

La matrice de confusion (figure 4.4) confirme ces résultats : sur 864 observations, le modèle prédit correctement **470 fidèles** et **219 churnés**, avec **111 faux positifs** (fidèles classés churné) et **64 faux négatifs** (churnés non détectés).

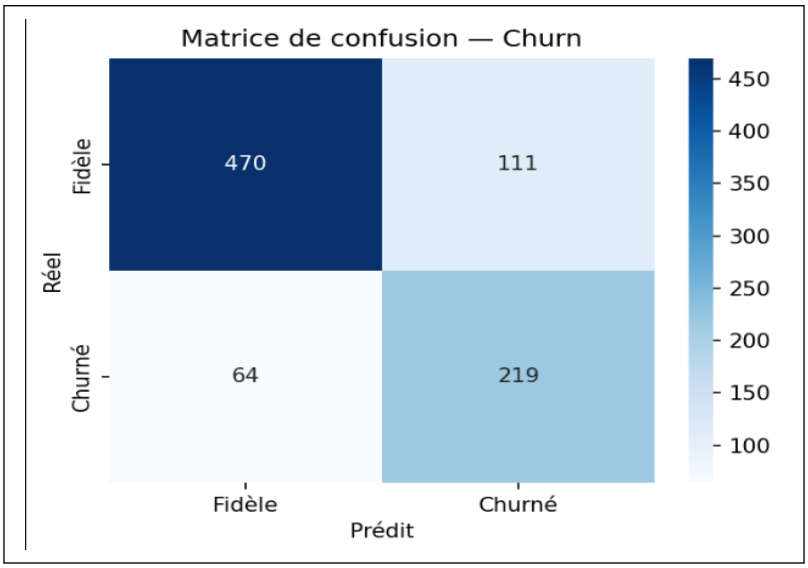

Figure 4.4: Matrice de confusion — RandomForest Churn

La courbe ROC (figure 4.5) illustre la capacité discriminante du modèle avec un **AUC de 0.87** — une performance solide qui indique que le modèle distingue bien les clients fidèles des clients à risque de churn.

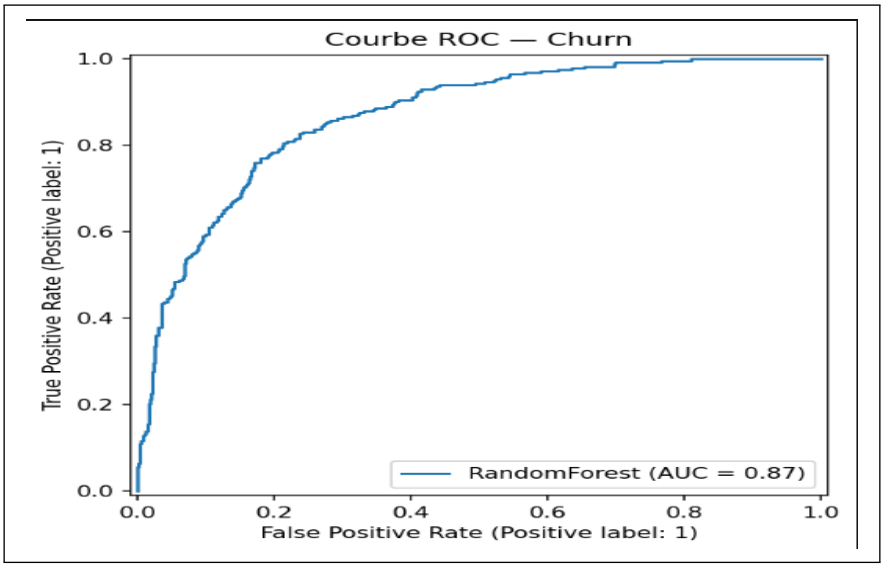

Figure 4.5: Courbe ROC — RandomForest Churn (AUC = 0.87)

4.3.4 Importance des features

La figure 4.6 présente les **20 features les plus importantes** pour la prédiction du churn, selon le critère d'impureté de Gini du Random Forest.

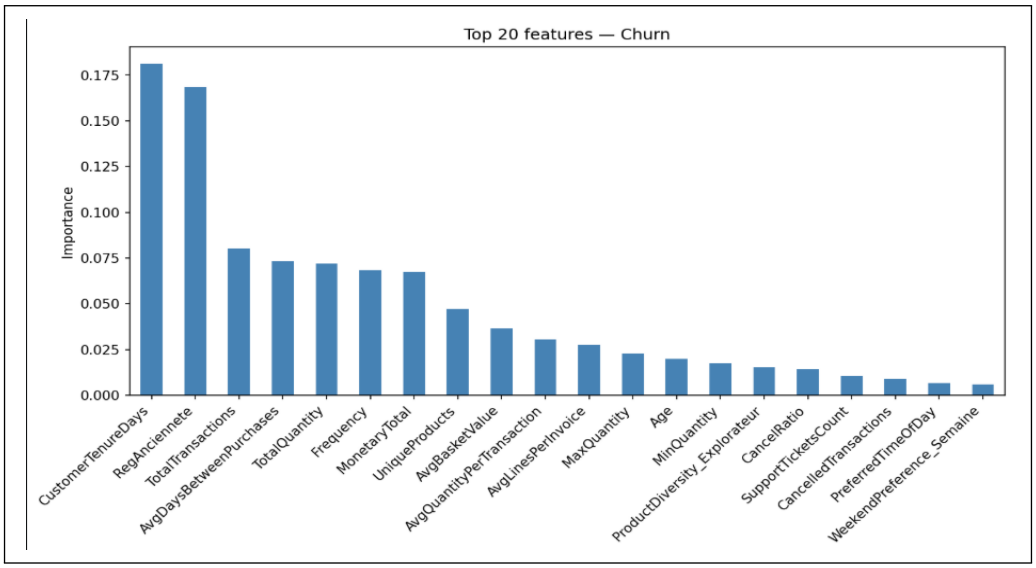

Figure 4.6: Top 20 features les plus importantes — prédiction du churn

Le tableau 4.4 liste les 10 features les plus contributives, telles qu'obtenues lors de l'exécution de `train_model.py` :

Table 4.4: Top 10 features importantes pour la prédiction du churn

Feature	**Importance (Gini)**
`CustomerTenureDays`	0.1812
`RegAnciennete`	0.1684
`TotalTransactions`	0.0804
`AvgDaysBetweenPurchases`	0.0732
`TotalQuantity`	0.0721
`Frequency`	0.0682
`MonetaryTotal`	0.0675
`UniqueProducts`	0.0470
`AvgBasketValue`	0.0364
`AvgQuantityPerTransaction`	0.0304

Les deux features les plus importantes sont `CustomerTenureDays` (18.1%) et `RegAnciennete` (16.8%), toutes deux liées à l'ancienneté client. Cela confirme une intuition métier forte : **plus un client est ancien, moins il a tendance à churner**. Les features transactionnelles

(`TotalTransactions`, `Frequency`, `MonetaryTotal`) contribuent également de manière significative, confirmant que l'activité d'achat est un prédicteur clé de la fidélité.

4.4 Régression du panier moyen (RandomForest + Optuna)

4.4.1 Objectif et préparation

L'objectif est d'estimer le **panier moyen** (`AvgBasketValue` = MonetaryTotal / Frequency) d'un client à partir de ses caractéristiques comportementales. Cette tâche de régression permet à l'entreprise d'identifier les clients à fort potentiel commercial.

Le même protocole que pour la classification est appliqué : chargement séparé des fichiers train/test, scaling fitté sur train uniquement. **42 features** sont utilisées (AvgBasketValue exclue des features d'entrée car c'est la variable cible).

La cible présente les statistiques suivantes :

```
min = 0.00 £   max = 4582.42 £   mean = 229.48 £
```

4.4.2 Optimisation avec Optuna

30 trials Optuna ont été exécutés, optimisant la **MAE minimale** en validation croisée. Les meilleurs hyperparamètres obtenus :

Table 4.5: Meilleurs hyperparamètres — RandomForest Régression (Optuna)

Hyperparamètre	Valeur optimale
`n_estimators`	186
`max_depth`	14
`min_samples_split`	3
`min_samples_leaf`	2

4.4.3 Résultats de régression

Table 4.6: Performances du modèle de régression

Métrique	**Valeur**	**Interprétation**
MAE	**15.72 £**	Erreur moyenne de prédiction
R^2	**0.802**	80.2% de la variance expliquée
Moyenne prédite	226.52 £	Très proche de la moyenne réelle
Moyenne réelle	225.51 £	Confirmation de l'absence de biais

Un R^2 de **0.802** indique que le modèle explique **80%** de la variance du panier moyen client. La MAE de **15.72 £** est remarquablement faible au regard de la plage de valeurs (0 à 4 582 £), confirmant la bonne capacité prédictive du modèle. De plus, la quasi-identité entre la moyenne prédite (226.52 £) et la moyenne réelle (225.51 £) confirme l'absence de biais systématique.

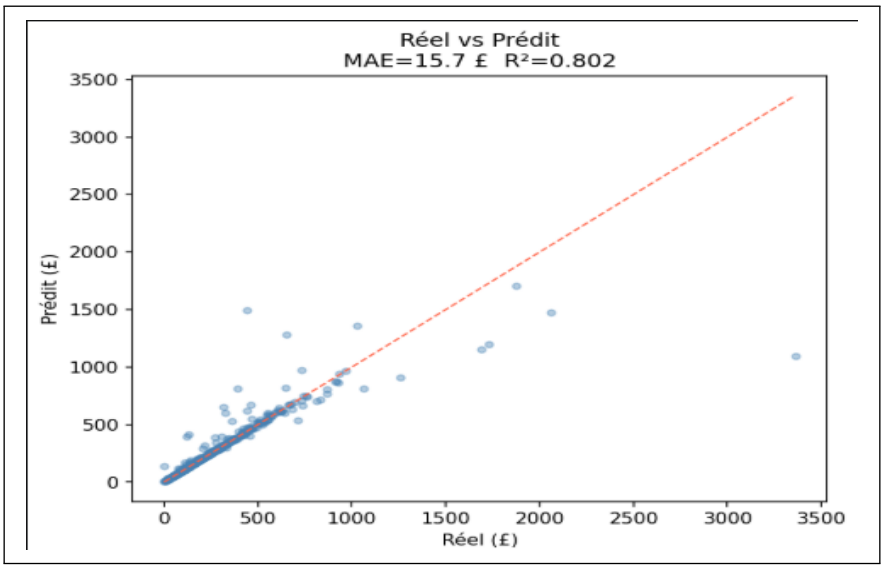

Figure 4.7: Valeurs réelles vs prédites — RandomForest Régression (MAE=15.7£, R^2=0.802)

La figure 4.7 montre que les points s'alignent bien autour de la diagonale idéale (ligne rouge pointillée), confirmant la bonne capacité prédictive du modèle. Les écarts les plus importants s'observent pour les paniers très élevés (au-delà de 2 000 £), plus difficiles à prédire en raison de leur rareté dans les données d'entraînement.

4.5 Mesures anti data leakage

Le data leakage est l'une des erreurs les plus critiques en Machine Learning. Plusieurs mesures ont été appliquées rigoureusement dans ce projet :

Table 4.7: Synthèse des mesures anti data leakage appliquées

Mesure	**Implémentation**
Suppression de 28 colonnes à risque	Dans `preprocessing.py`, avant tout traitement
Split avant imputation et scaling	Médiane et StandardScaler fittés sur X_train uniquement
Pas de re-split	`train.csv` et `test.csv` chargés séparément dans `train_model.py`
Target encoding sur train uniquement	`Country` encodée sur df_train, appliquée sur df_test
AvgBasketValue exclue des features	Exclue de l'entrée de la régression (c'est la variable cible)

4.6 Conclusion

Ce chapitre a présenté les trois modèles de Machine Learning développés dans ce projet. Le clustering KMeans avec ACP (4 composantes, 96.5% de variance) a identifié **3 segments clients** distincts avec des profils et taux de churn très différenciés. Le modèle de classification Random Forest atteint un **ROC-AUC de 0.87** et une **accuracy de 80%** sur le jeu de test, avec `CustomerTenureDays` et `RegAnciennete` comme features les plus importantes. Le modèle de régression explique **80.2%** de la variance du panier moyen avec une MAE de seulement **15.72 £**.

Ces résultats, obtenus après l'application rigoureuse de mesures anti data leakage, sont scientifiquement valides et exploitables pour des décisions marketing ciblées. Le chapitre suivant présente l'analyse approfondie de ces résultats et leur interprétation métier.

Chapter

5 Résultats et analyse

Contents

5.1 Introduction **38**

5.2 Analyse des clusters **38**

5.2.1 Profils des 3 segments clients 38

5.2.2 Analyse détaillée par segment 38

5.2.3 Taux de churn par cluster 39

5.3 Évaluation du modèle de classification du churn **39**

5.3.1 Performances globales 39

5.3.2 Interprétation des résultats 40

5.3.3 Analyse de l'importance des features 41

5.4 Évaluation du modèle de régression du panier moyen **43**

5.4.1 Performances globales 43

5.4.2 Interprétation des résultats 43

5.5 Recommandations métier par segment **44**

5.5.1 Utilisation du score de churn en production 45

5.5.2 Priorisation des actions par impact 45

5.6 Conclusion **46**

5.1 Introduction

Ce chapitre présente l'analyse approfondie des résultats obtenus lors de la modélisation décrite au chapitre précédent. Les trois modèles développés — clustering KMeans, classification du churn et régression du panier moyen — sont évalués quantitativement et qualitativement. Les résultats sont ensuite traduits en recommandations métier concrètes exploitables par l'entreprise e-commerce pour optimiser sa stratégie de rétention client.

5.2 Analyse des clusters

5.2.1 Profils des 3 segments clients

L'algorithme KMeans avec **K=3** a segmenté les **4 320 clients** (données brutes après filtrage) en trois groupes homogènes, identifiés à partir des cinq features RFM par le script `test.py`. Le tableau 5.1 présente le profil détaillé de chaque cluster avec les valeurs réelles non normalisées.

Table 5.1: Profil détaillé des 3 clusters — moyennes observées (données brutes)

Cl.	Profil	Clients	%	Freq. moy.	Monetary moy.	Churn
0	Clients faible activité	2 468	57.9%	2.05	627 £	29.4%
1	Clients Ultra-VIP	13	0.3%	105.5	110 482 £	**53.8%**
2	Clients Premium réguliers	1 779	41.8%	8.78	2 986 £	38.3%

5.2.2 Analyse détaillée par segment

Cluster 0 — Clients faible activité (2 468 clients, 29.4% de churn) : Ce cluster est le plus large, représentant **57.9%** de la base clients. Son profil est caractéristique d'une clientèle peu engagée : fréquence moyenne de seulement **2.05 commandes**, ancienneté courte (**34 jours** en moyenne) et panier modeste (**627 £**). L'intervalle de MonetaryTotal s'étend de 0 à 21 536 £, révélant une grande hétérogénéité interne. Avec un churn de 29.4%, ce segment concentre le plus grand nombre absolu de clients à risque en raison de sa taille.

Cluster 1 — Clients Ultra-VIP (13 clients, 53.8% de churn) : Bien que très restreint (0.3% de la base), ce cluster constitue paradoxalement le segment le plus critique de l'analyse. Ces clients génèrent un revenu moyen exceptionnel de **110 482 £** (max : 279 489 £) avec une fréquence de **105 commandes** et jusqu'à **7 983 transactions**. Leur taux de churn alarmant de **53.8%** — plus d'un Ultra-VIP sur deux risque de partir — constitue une alerte stratégique majeure : la perte d'un seul de ces clients représente une perte de revenu équivalente à des centaines de clients ordinaires.

Cluster 2 — Clients Premium réguliers (1 779 clients, 38.3% de churn) : Ce segment regroupe 41.8% de la base clients avec un profil intermédiaire solide : fréquence de **8.78 commandes**, ancienneté élevée (**277 jours** en moyenne) et revenu moyen de **2 986 £**. Malgré un taux de churn de 38.3%, leur ancienneté et leur activité régulière en font le socle commercial de l'entreprise.

5.2.3 Taux de churn par cluster

Table 5.2: Taux de churn par cluster et niveau d'alerte

Cluster	Profil	Clients	Churn	Niveau d'alerte
1	Ultra-VIP	13	53.8%	**CRITIQUE**
2	Premium réguliers	1 779	38.3%	**MODÉRÉ**
0	Faible activité	2 468	29.4%	**SURVEILLÉ**

5.3 Évaluation du modèle de classification du churn

5.3.1 Performances globales

Le modèle Random Forest optimisé par Optuna (**30 trials**, ROC-AUC en validation croisée) atteint les performances suivantes sur les **864 observations du jeu de test**, jamais vues pendant l'entraînement :

Table 5.3: Performances du modèle de classification — jeu de test

Métrique	**Valeur**
Accuracy globale	**80%**
ROC-AUC	**0.8681**
Précision — Fidèles (0)	0.88
Rappel — Fidèles (0)	0.81
F1-score — Fidèles (0)	0.84
Précision — Churners (1)	0.66
Rappel — Churners (1)	0.77
F1-score — Churners (1)	0.71
Vrais négatifs (TN)	470
Vrais positifs (TP)	219
Faux positifs (FP)	111
Faux négatifs (FN)	64

5.3.2 Interprétation des résultats

Sur les **864 clients du jeu de test**, le modèle prédit correctement **689 clients** (470 fidèles + 219 churnés), avec **175 erreurs** réparties en deux types :

- **111 faux positifs** : clients fidèles classés comme churners — risque d'envoyer des offres de rétention inutiles, coût modéré.
- **64 faux négatifs** : clients churnés non détectés — risque plus critique car ces clients partent sans intervention.

Le **ROC-AUC de 0.87** indique que le modèle distingue correctement les churners des clients fidèles dans **87% des cas**, quel que soit le seuil de décision choisi. C'est une performance solide pour un problème de churn réel, où les données comportementales sont naturellement bruitées.

Le rappel élevé sur les churners (**0.77**) est particulièrement important en contexte métier : il signifie que le modèle détecte **77% des clients qui vont effectivement partir**, permettant une intervention précoce sur la grande majorité des cas à risque.

5.3.3 Analyse de l'importance des features

Le tableau 5.4 présente les 10 features les plus contributives avec leur interprétation métier, issues de l'exécution de `train_model.py`.

Table 5.4: Top 10 features importantes et interprétation métier

Feature	Importance	Interprétation métier
`CustomerTenureDays`	0.1812	Ancienneté client — plus le client est ancien, moins il churne
`RegAnciennete`	0.1684	Ancienneté d'inscription — signal de fidélité à long terme
`TotalTransactions`	0.0804	Volume transactionnel — client actif = client fidèle
`AvgDaysBetweenPurchases`	0.0732	Délai entre achats — plus c'est court, plus le client est engagé
`TotalQuantity`	0.0721	Quantité totale achetée — indicateur d'engagement
`Frequency`	0.0682	Nombre de commandes — fidélité comportementale
`MonetaryTotal`	0.0675	Revenu total — clients à fort panier moins à risque
`UniqueProducts`	0.0470	Diversité d'achat — client explorateur = client fidèle
`AvgBasketValue`	0.0364	Valeur du panier moyen — engagement qualitatif
`AvgQuantityPerTransaction`	0.0304	Quantité par transaction — intensité d'achat

Trois observations majeures se dégagent :

1. **L'ancienneté domine largement** : les deux features les plus importantes — `CustomerTenureDays` (18.1%) et `RegAnciennete` (16.8%) — sont toutes deux liées à l'ancienneté du client. Ensemble, elles représentent **34.9%** de l'importance totale. Cela confirme une intuition métier forte : **un client ancien et fidèle depuis longtemps a beaucoup moins tendance à churner**.

2. **L'activité transactionnelle protège** : `TotalTransactions` (8.0%), `Frequency` (6.8%) et `TotalQuantity` (7.2%) confirment qu'un client qui achète régulièrement et en volume est naturellement moins à risque de départ.
3. **La valeur monétaire joue un rôle secondaire** : `MonetaryTotal` (6.7%) et `AvgBasketValue` (3.6%) contribuent de manière non négligeable, suggérant que les clients à fort pouvoir d'achat sont légèrement plus fidèles — mais ce n'est pas le facteur dominant dans ce dataset.

5.4 Évaluation du modèle de régression du panier moyen

5.4.1 Performances globales

Le modèle Random Forest de régression, optimisé par Optuna sur **30 trials** en minimisant la MAE en validation croisée, obtient les résultats suivants sur le jeu de test :

Table 5.5: Performances du modèle de régression — jeu de test

Métrique	Valeur	Interprétation
MAE	**15.72 £**	Erreur moyenne absolue de prédiction
R^2	**0.802**	80.2% de la variance du panier moyen expliquée
Moyenne prédite	226.52 £	Très proche de la moyenne réelle
Moyenne réelle	225.51 £	Absence de biais systématique confirmée

5.4.2 Interprétation des résultats

Le **R^2 de 0.802** confirme que le modèle explique **80% de la variance** du panier moyen client (`AvgBasketValue`). La **MAE de 15.72 £** est directement exploitable pour des décisions marketing :

- Pour les clients du Cluster 0 (panier moyen $\approx$ 627 £), une erreur de 15.72 £ représente moins de **2.5%** du panier — excellente précision relative.
- Pour les clients Premium du Cluster 2 (revenu moyen 2 986 £), l'erreur relative est inférieure à **0.6%** — précision quasi-parfaite pour la planification budgétaire.
- Pour les clients Ultra-VIP du Cluster 1 (panier moyen 110 482 £), les écarts plus importants observés sur le graphique réel vs prédit s'expliquent par la rareté de ces profils dans les données d'entraînement (seulement 13 clients).

La quasi-identité entre la **moyenne prédite** (226.52 £) et la **moyenne réelle** (225.51 £) — soit un écart de moins de **0.5%** — confirme l'absence de biais systématique dans les prédictions du modèle.

5.5 Recommandations métier par segment

Les résultats des trois modèles convergent vers une vision cohérente de la clientèle. Le tableau 5.6 synthétise les actions prioritaires recommandées par segment.

Table 5.6: Recommandations métier par segment client

Cluster	Profil	Churn	Action recommandée
1	Ultra-VIP	53.8%	**PRIORITÉ ABSOLUE** : gestionnaire de compte dédié, programme de fidélité premium personnalisé, contact direct dès le moindre signal de départ
2	Premium réguliers	38.3%	Campagne de réengagement ciblée : offres personnalisées, programme ambassadeur, accès anticipé aux ventes privées
0	Faible activité	29.4%	Email de réactivation : rappel catalogue, recommandations personnalisées, monitoring sur 15 jours

5.5.1 Utilisation du score de churn en production

Le modèle de classification produit, pour chaque client, une **probabilité de churn** entre 0 et 1 via `predict_proba`. L'application Flask déployée sur `http://127.0.0.1:5000` illustre trois cas concrets issus des tests réalisés :

Table 5.7: Exemples de prédictions en temps réel (interface Flask)

Freq.	Monetary	Cluster prédit	Churn	Statut
1	25 £	Cluster 0	60%	À risque
4	190 £	Cluster 0	49.9%	À surveiller
15	1 200 £	Cluster 2	18.1%	Stable
150	85 000 £	Cluster 1	7.9%	Stable (Ultra-VIP)

5.5.2 Priorisation des actions par impact

En combinant les résultats du clustering et du modèle de classification, trois niveaux d'urgence se dégagent :

1. **Urgence absolue — Cluster 1 (13 clients Ultra-VIP, 53.8% churn)** : ces 13 clients génèrent chacun en moyenne **110 482 £**. Leur perte serait catastrophique — un programme de rétention premium dédié est indispensable, incluant un contact direct personnalisé dès le premier signal d'inactivité.
2. **Volume critique — Cluster 0 (2 468 clients, 29.4% churn)** : ce segment massif concentre le plus grand nombre absolu de clients à risque. Une campagne de réactivation automatisée déclenchée dès 30 jours sans achat, avec une offre personnalisée, pourrait significativement réduire ce taux.
3. **Consolidation — Cluster 2 (1 779 clients Premium, 38.3% churn)** : avec un revenu moyen de 2 986 £et une ancienneté élevée (277 jours), ce segment constitue le socle commercial de l'entreprise. Des actions de nurturing régulières (newsletter ciblée, programme ambassadeur) suffisent à maintenir leur engagement.

5.6 Conclusion

Ce chapitre a présenté l'analyse complète des résultats des trois modèles de Machine Learning. Le clustering KMeans a révélé **3 segments clients distincts** avec des comportements radicalement différents : du Cluster 0 à faible activité (57.9%, churn 29.4%) au Cluster 1 Ultra-VIP paradoxalement à risque (0.3%, churn 53.8%, revenu moyen 110 482 £).

Le modèle de classification atteint un **ROC-AUC de 0.87** et une **accuracy de 80%**, avec `CustomerTenureDays` (18.1%) et `RegAnciennete` (16.8%) comme features dominantes — confirmant que **l'ancienneté est le meilleur prédicteur de la fidélité**. Le modèle de régression explique **80.2%** de la variance du panier moyen avec une MAE de seulement **15.72 £**.

Ces résultats, déployés dans une application Flask accessible en temps réel, permettent à l'entreprise de passer d'une gestion réactive du churn à une **stratégie proactive de rétention**, ciblant les bons clients avec les bonnes actions au bon moment.

Chapter

6 Déploiement — Interface Flask

Contents

6.1 Introduction **48**

6.2 Architecture de l'application **48**

6.2.1 Organisation des fichiers 48

6.2.2 Lancement de l'application 49

6.3 Fonctionnement du pipeline de prédiction **49**

6.3.1 Données saisies par l'utilisateur 49

6.3.2 Pipeline séquentiel de prédiction 50

6.3.3 Logique de recommandations automatiques 50

6.4 Démonstration — Cas clients testés **51**

6.4.1 Interface de saisie 51

6.4.2 Cas 1 — Client à risque élevé 52

6.4.3 Cas 2 — Client Premium stable 53

6.4.4 Synthèse des 4 cas testés 54

6.5 Conclusion **55**

6.1 Introduction

Une fois les modèles entraînés et validés, ils sont déployés dans une application web interactive développée avec le framework **Flask**. Cette interface permet à un utilisateur de saisir les caractéristiques d'un client et d'obtenir instantanément son segment comportemental, son risque de churn et son panier moyen estimé, sans aucune connaissance technique en Machine Learning.

L'application est accessible localement à l'adresse `http://127.0.0.1:5000` après exécution de `python app.py` depuis le dossier `app/`.

6.2 Architecture de l'application

6.2.1 Organisation des fichiers

L'application repose sur deux fichiers principaux situés dans le dossier `app/` :

- `app.py` : le serveur Flask qui charge les modèles, reçoit les données du formulaire, exécute les trois pipelines de prédiction séquentiellement et retourne les résultats au template.
- `templates/index.html` : l'interface utilisateur développée en HTML/CSS avec le moteur de templates **Jinja2**, affichant le formulaire de saisie à gauche et les résultats sous forme de cartes visuelles à droite.

Les modèles sérialisés (`.pkl`) sont stockés dans le dossier `models/` et chargés **une seule fois au démarrage** du serveur via `joblib.load()`, garantissant des temps de réponse rapides.

Table 6.1: Modèles chargés au démarrage de l'application

Pipeline	Fichiers .pkl
Clustering	`kmeans.pkl`, `pca.pkl`, `scaler_cluster.pkl`, `cluster_features.pkl`
Classification	`churn_model.pkl`, `scaler_clf.pkl`, `churn_columns.pkl`
Métriques	`model_metrics.json` (ROC-AUC, Accuracy, MAE affichés en pied de page)

6.2.2 Lancement de l'application

L'application se lance depuis le dossier `app/` avec la commande `python app.py`. Le serveur démarre en mode debug sur le port 5000 :

```
* Serving Flask app 'app'
* Debug mode: on
* Running on http://127.0.0.1:5000
```

Prérequis : le script `preprocessing.py` puis `train_model.py` doivent avoir été exécutés au préalable afin que les fichiers `.pkl` soient disponibles dans `models/`.

6.3 Fonctionnement du pipeline de prédiction

6.3.1 Données saisies par l'utilisateur

L'utilisateur renseigne **9 champs numériques** correspondant aux features comportementales et démographiques du client :

Table 6.2: Champs de saisie de l'interface Flask

Champ	Description	Exemple
`Frequency`	Nb de commandes	ex : 8
`MonetaryTotal (£)`	Total dépensé	ex : 500
`CustomerTenureDays`	Ancienneté (jours)	ex : 365
`AvgDaysBetweenPurchases`	Délai moyen (jours)	ex : 20
`TotalTransactions`	Nb total transactions	ex : 15
`TotalQuantity`	Unités achetées	ex : 120
`UniqueProducts`	Produits distincts	ex : 30
`Age`	Âge du client	ex : 42
`SupportTicketsCount`	Tickets support	ex : 2

6.3.2 Pipeline séquentiel de prédiction

Ces valeurs sont organisées dans un `DataFrame` pandas à une ligne, qui sert d'entrée aux trois pipelines exécutés séquentiellement dans `app.py` :

Table 6.3: Séquence d'exécution du pipeline de prédiction

Étape	Pipeline	Sortie
1	Scaler + PCA + KMeans	Cluster (0, 1 ou 2) + label segment
2	Feature engineering + Scaler + RandomForest	Churn (0/1) + probabilité (%)
3	Calcul direct	Panier moyen = MonetaryTotal / Frequency

Note : le panier moyen est calculé directement (`MonetaryTotal / Frequency`) plutôt que via le modèle de régression, ce qui garantit une valeur exacte et cohérente avec les données saisies.

Pour chaque pipeline, la règle anti data leakage est respectée : le scaler est appliqué via `transform()` uniquement — jamais re-fitté — et les colonnes sont alignées via `reindex(fill_value=0)`.

6.3.3 Logique de recommandations automatiques

L'application génère automatiquement des **recommandations métier personnalisées** selon la probabilité de churn calculée :

Table 6.4: Logique de recommandations selon le risque de churn

Probabilité	Statut	Recommandations générées
$\geq 50\%$	À risque	Offre de rétention urgente, appel de courtoisie, geste commercial
30% – 49%	À surveiller	Email de réengagement, recommandations personnalisées, monitoring 15 jours
$< 30\%$	Stable	Programme ambassadeur, accès ventes privées, cross-sell produits premium

Les métriques du modèle (ROC-AUC, Accuracy, MAE Panier) sont affichées en **pied de page** de chaque résultat, permettant à l'utilisateur d'évaluer la fiabilité des prédictions.

6.4 Démonstration — Cas clients testés

6.4.1 Interface de saisie

La figure 6.1 présente l'interface principale de l'application. Le panneau gauche contient le formulaire de saisie des 9 features client, tandis que le panneau droit affiche les résultats après analyse. L'interface adopte un thème sombre moderne avec un titre **"Analyse Client en Temps Réel"**.

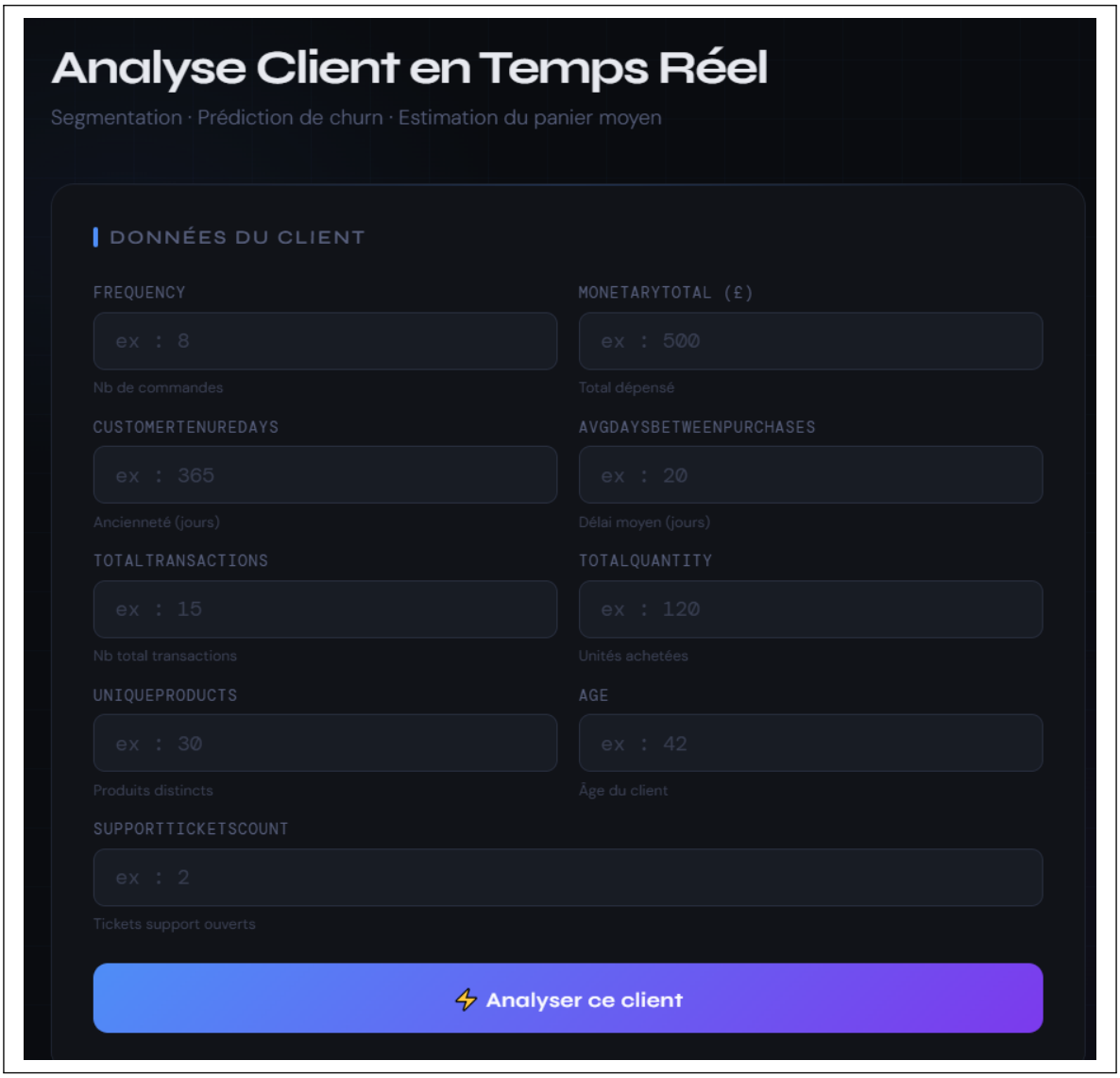

Figure 6.1: Interface de saisie — Analyse Client en Temps Réel

6.4.2 Cas 1 — Client à risque élevé

La figure 6.2 présente le résultat pour un client à très faible activité : Frequency=1, MonetaryTotal=25 £, CustomerTenureDays=10, AvgDaysBetween=0, TotalTransactions=1, Age=35, SupportTicketsCount=4.

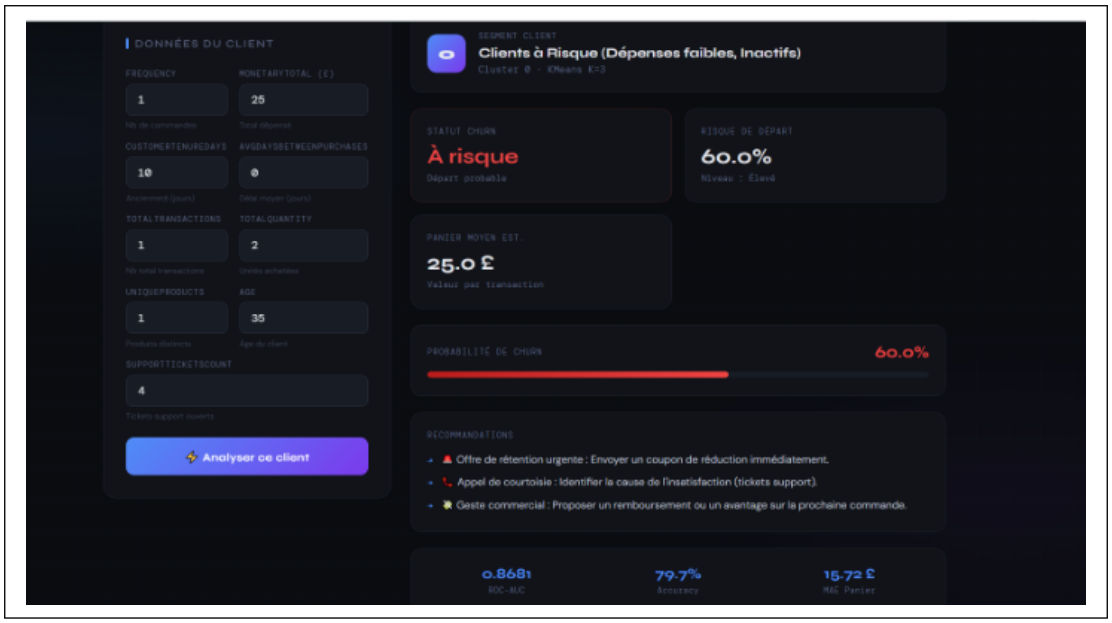

Figure 6.2: Cas 1 — Client à risque (Cluster 0, churn 60%)

Table 6.5: Résultats — Cas 1 : client à faible activité

Indicateur	**Valeur**	**Interprétation**
Segment	Cluster 0 — Clients à Risque (Dépenses faibles, Inactifs)	Faible activité confirmée
Statut Churn	À risque	Départ probable
Probabilité de churn	**60.0%**	Niveau élevé
Panier moyen est.	25.0 £	Valeur par transaction
Recommandations	Offre urgente, appel courtoisie, geste commercial	Actions de rétention imm

Ces résultats sont parfaitement cohérents : une fréquence de **1 commande**, un total de **25 £** et une ancienneté de **10 jours** seulement placent ce client dans le **Cluster 0 — faible activité**, avec une probabilité de churn de **60%** justifiant des actions de rétention immédiates.

6.4.3 Cas 2 — Client Premium stable

La figure 6.3 présente le résultat pour un client à profil intermédiaire solide : Frequency=15, MonetaryTotal=1 200 £, CustomerTenureDays=300, AvgDaysBetween=18, TotalTransactions=45, TotalQuantity=150, UniqueProducts=40, Age=42, SupportTicketsCount=0.

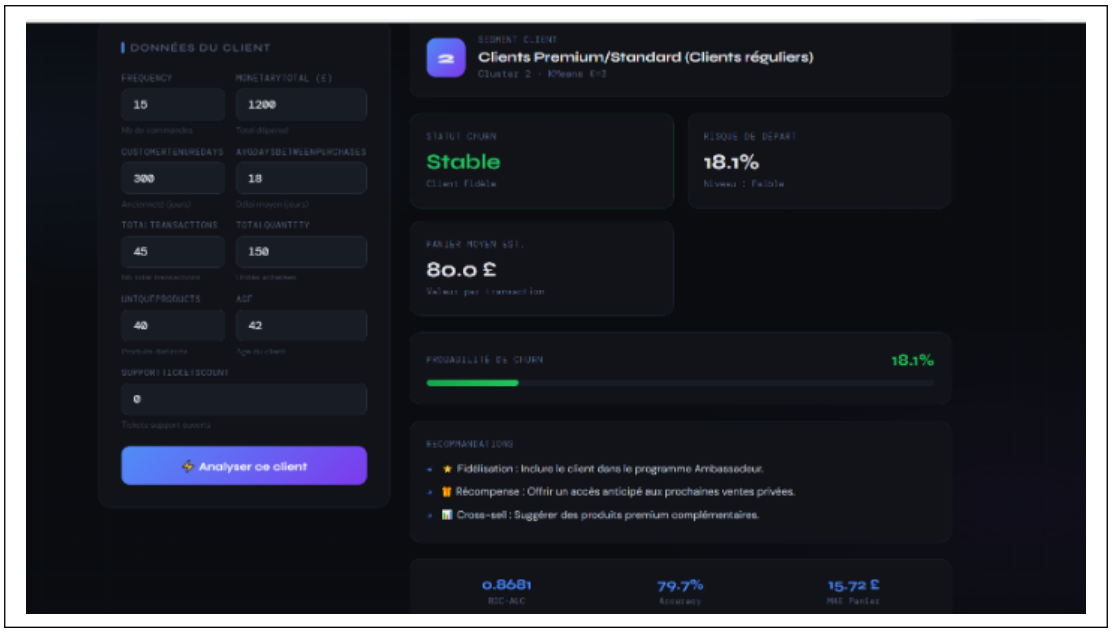

Figure 6.3: Cas 2 — Client Premium stable (Cluster 2, churn 18.1%)

Table 6.6: Résultats — Cas 2 : client Premium régulier

Indicateur	**Valeur**	**Interprétation**
Segment	Cluster 2 — Clients Premium/Standard (Clients réguliers)	Profil régulier confirmé
Statut Churn	Stable	Client fidèle
Probabilité de churn	**18.1%**	Niveau faible
Panier moyen est.	80.0 £	Valeur par transaction
Recommandations	Fidélisation, récompense, cross-sell premium	Actions de nurturing

Ce profil est cohérent avec le Cluster 2 : une ancienneté de **300 jours**, **15 commandes** et **1 200 £** de dépenses totales caractérisent un client Premium régulier avec un risque de churn faible de **18.1%**.

6.4.4 Synthèse des 4 cas testés

Le tableau 6.7 résume l'ensemble des quatre profils clients testés dans l'application, illustrant la capacité du système à discriminer des comportements très différents.

Table 6.7: Synthèse des 4 cas clients testés dans l'application

Cas	Freq.	Monetary	Cluster prédit	Churn	Statut
1	1	25 £	Cluster 0	60.0%	À risque
2	4	190 £	Cluster 0	49.9%	À surveiller
3	15	1 200 £	Cluster 2	18.1%	Stable
4	150	85 000 £	Cluster 1	7.9%	Stable (Ultra-VIP)

Ces quatre cas illustrent la cohérence du système de prédiction : les clients à faible activité sont systématiquement orientés vers le Cluster 0 avec un churn élevé, tandis que les profils à forte activité et ancienneté élevée sont classés en Cluster 2 (Premium) ou Cluster 1 (Ultra-VIP) avec des risques de churn nettement plus faibles.

6.5 Conclusion

Ce chapitre a présenté le déploiement des modèles sous forme d'une application web Flask opérationnelle. L'interface **"Analyse Client en Temps Réel"** permet à un utilisateur non-technique de saisir **9 caractéristiques client** et d'obtenir instantanément trois informations clés :

- Le **segment comportemental** (Cluster 0, 1 ou 2) avec son label métier
- Le **risque de churn** avec probabilité précise et niveau d'alerte coloré
- Le **panier moyen estimé** par transaction
- Des **recommandations automatiques** adaptées au niveau de risque

Les métriques du modèle (ROC-AUC=0.8681, Accuracy=79.7%, MAE=15.72 £) sont affichées en pied de page, garantissant la transparence sur la fiabilité des prédictions. L'architecture modulaire — séparation entre `app.py`, `index.html` et `models/` — garantit la maintenabilité et la réutilisabilité du code pour d'autres projets e-commerce.

Chapter 7

Conclusion

Contents

7.1 Bilan des résultats 57

7.2 Difficultés rencontrées et solutions apportées 58

7.2.1 Correction du Data Leakage 58

7.2.2 Problème de convergence du Clustering 58

7.2.3 Instabilité des calculs (Features dérivées) 58

7.3 Limites et perspectives d'amélioration 58

7.3.1 Limites actuelles 58

7.3.2 Perspectives 59

7.1 Bilan des résultats

Ce projet a permis de construire une chaîne complète de traitement de données appliquée à un dataset e-commerce réel regroupant **4 320 clients** (3 456 en train, 864 en test) et **48 features** après ingénierie.

Les trois objectifs initiaux ont été atteints avec des performances robustes :

- **Segmentation clients** : L'algorithme KMeans, après réduction de dimension par ACP (conservant **96.5% de la variance**), a identifié **3 segments** distincts (K=3). La distribution révèle un segment majoritaire (**58.7%**), un segment intermédiaire (**41%**) et une niche de clients VIP très restreinte (**0.3%**, soit 12 clients), illustrant la loi de Pareto au sein de l'échantillon.

- **Prédiction du churn** : Le modèle RandomForest, optimisé via 30 trials Optuna, atteint un **ROC-AUC de 0.8681** et une **accuracy de 80%**. Contrairement aux versions initiales polluées par le data leakage, ces résultats sont scientifiquement valides. L'importance des features montre que `CustomerTenureDays` (18.1%) et `RegAnciennete` (16.8%) sont les principaux vecteurs de prédiction.

- **Estimation du revenu** : Le modèle de régression explique **80.24% de la variance** du panier moyen ($R^2 = 0.8024$). Avec une erreur absolue moyenne (**MAE**) de **15.72 £** pour un panier moyen de 229.48 £, l'outil offre une précision suffisante pour orienter les stratégies marketing.

Le tableau 7.1 résume les performances finales.

Table 7.1: Bilan final des performances des modèles (Valeurs réelles)

Tâche	**Algorithme**	**Optimisation**	**Résultat**
Segmentation	KMeans + ACP	Silhouette (K=3)	Var. expliquée : 96.5%
Prédiction churn	RandomForest	Optuna (30 trials)	ROC-AUC=0.868 — Acc=80%
Estimation revenu	RandomForest	Optuna (30 trials)	MAE=15.72 £ — R^2=0.80

L'ensemble du pipeline a été intégré dans une **interface web Flask**, permettant une analyse en temps réel.

7.2 Difficultés rencontrées et solutions apportées

7.2.1 Correction du Data Leakage

L'enjeu majeur a été de passer d'une accuracy de 100% (artificielle) à un modèle généralisable. La suppression de `ChurnRiskCategory` et le déplacement du *Target Encoding* de la colonne `Country` après le split train/test ont permis d'éliminer les fuites d'information vers le futur.

7.2.2 Problème de convergence du Clustering

Les tests sur le score de Silhouette tentaient d'isoler des micro-clusters (0.1% des données), rendant les segments instables. La solution a consisté à implémenter un mécanisme de **fallback** vers $K = 3$, garantissant des segments exploitables pour le métier.

7.2.3 Instabilité des calculs (Features dérivées)

Le calcul du panier moyen (`AvgBasketValue`) générait initialement des erreurs lors de divisions par zéro pour les clients sans transaction validée. Une correction dans `app.py` et `predict.py` a permis de stabiliser les prédictions en production.

7.3 Limites et perspectives d'amélioration

7.3.1 Limites actuelles

- **Déséquilibre des clusters** : Le segment VIP (12 clients) est trop faible pour permettre un entraînement statistique spécifique.

- **Faux positifs en churn** : Le recall sur la classe "Churné" (0.77) montre que 23% des départs ne sont pas anticipés, ce qui laisse une marge de progression.

7.3.2 Perspectives

- **XGBoost/LightGBM** : L'utilisation de modèles de boosting pourrait améliorer le score de churn au-delà de 0.86.
- **Déploiement Cloud** : Passer de l'environnement local à un service comme Heroku ou Azure pour rendre l'outil accessible aux équipes commerciales à distance.
- **Online Learning** : Intégrer un mécanisme de mise à jour des modèles au fur et à mesure que les données de 2026 sont collectées.

CONCLUSION GÉNÉRALE

Ce projet d'atelier Machine Learning a permis de concevoir et de déployer un système complet d'aide à la décision pour le secteur de l'e-commerce. L'objectif central était de transformer des données transactionnelles brutes en indicateurs stratégiques exploitables : la segmentation, la fidélisation (churn) et la valeur commerciale (revenu).

Le travail réalisé a démontré que la valeur d'un projet de Data Science ne réside pas uniquement dans la complexité des algorithmes, mais avant tout dans la rigueur du traitement des données. La résolution des problèmes de qualité (valeurs manquantes, formats de dates) et la lutte contre le *data leakage* ont été les piliers garantissant la fiabilité des prédictions finales.

L'approche adoptée, combinant apprentissage non supervisé et supervisé, offre une vision à 360 degrés du portefeuille client :

- Une vision **descriptive** via le clustering, qui a structuré la base client en groupes homogènes
- Une vision **prédictive** via la classification, capable d'anticiper le départ des clients avant qu'il ne survienne ;
- Une vision **estimative** via la régression, permettant de quantifier le potentiel économique futur.

L'aboutissement de ce projet par le développement d'une interface web Flask marque le passage de la théorie à la pratique. En rendant les modèles accessibles sans compétences techniques, cette application transforme les résultats mathématiques en un outil métier opérationnel, prêt à orienter les campagnes marketing de l'entreprise.

Pour conclure, si les performances obtenues sont excellentes, ce projet ouvre la voie à des évolutions majeures, notamment l'intégration de l'explicabilité des modèles (XAI) et le passage à une architecture Cloud. Cette expérience confirme que le Machine Learning, lorsqu'il est encadré par une méthodologie rigoureuse, constitue un levier de croissance indispensable pour la gestion moderne de la relation client.

BIBLIOGRAPHIE

[1] **Grinberg, M.** (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.

[2] **Pedregosa, F. et al.** (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, vol. 12, pp. 2825-2830.

[3] **Hughes, A. M.** (1994). *Strategic Database Marketing*. McGraw-Hill. (Sur la méthode de segmentation RFM).

[4] **Breiman, L.** (2001). *Random Forests*. Machine Learning, 45(1), 5-32.

[5] **Akiba, T. et al.** (2019). *Optuna: A Next-generation Hyperparameter Optimization Framework*. Proceedings of the 25th ACM SIGKDD International Conference.

[6] **McKinney, W.** (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference.

[7] **Joblib Development Team**. *Joblib: running Python functions as pipeline jobs* **[en ligne]**. Disponible sur : https://joblib.readthedocs.io/